

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

Пояснительная записка
к курсовой работе
по курсу «Программирование на языке Java»
на тему «Разработка многомодульного приложения на языке Java»

Выполнил:
Студент группы 23ВВП1
Гуреев Д. Р.

Приняла:
К.т.н. Юрова О. В.

Пенза 2026

Содержание

Введение.....	5
1. Постановка задачи.....	6
2. Выбор решения.....	7
3. Описание программы.....	8
3.1 Серверная часть.....	9
3.2 Клиентская часть.....	11
3.3 Часть логики	14
4. Описание способа организации пользовательского интерфейса	17
5. Описание результатов работы программы	18
Заключение	26
Список литературы	27
Приложение А. Листинг программы.....	28
Приложение Б. UML-диаграммы	102

Введение

В современном мире цифровые технологии и сетевое взаимодействие стали неотъемлемой частью повседневной жизни и индустрии развлечений. Возможность мгновенной передачи данных позволяет организовывать совместный досуг пользователей из любой точки мира в режиме реального времени.

Язык программирования Java выступает стандартом для создания надежного сетевого ПО и кроссплатформенных игровых приложений во всем мире. Благодаря развитой экосистеме, поддержке многопоточности и модульности, данный язык позволяет эффективно проектировать, тестировать и масштабировать сложные программные комплексы.

Для практического применения и закрепления принципов объектно-ориентированного программирования была выполнена разработка многомодульного клиент-серверного приложения — настольной логической игры, особенно распространенной на территории бывших южных республик СССР и на Кавказе – «Нарды».

1. Постановка задачи

Разработать систему многомодульных программ, клиент-серверной архитектуры, позволяющую играть в карточную игру пользователями в рамках локальной вычислительной сети.

Функции сервера: прием и передача информации от пользователя, сбор статистики по пользователям, работа с колодой и проверка комбинаций.

Функции клиента: графический интерфейс пользователю, сетевое взаимодействие с сервером. Операционная система: Windows; Среда разработки: IntelliJ IDEA; Язык программирования Java.

Используемые технологии:

- Java Collections Framework
- Механизм обработки исключительных ситуаций
- Java Stream API
- Java Multithreading
- Сетевое взаимодействие

Возможные варианты использования показаны на рисунке 1.

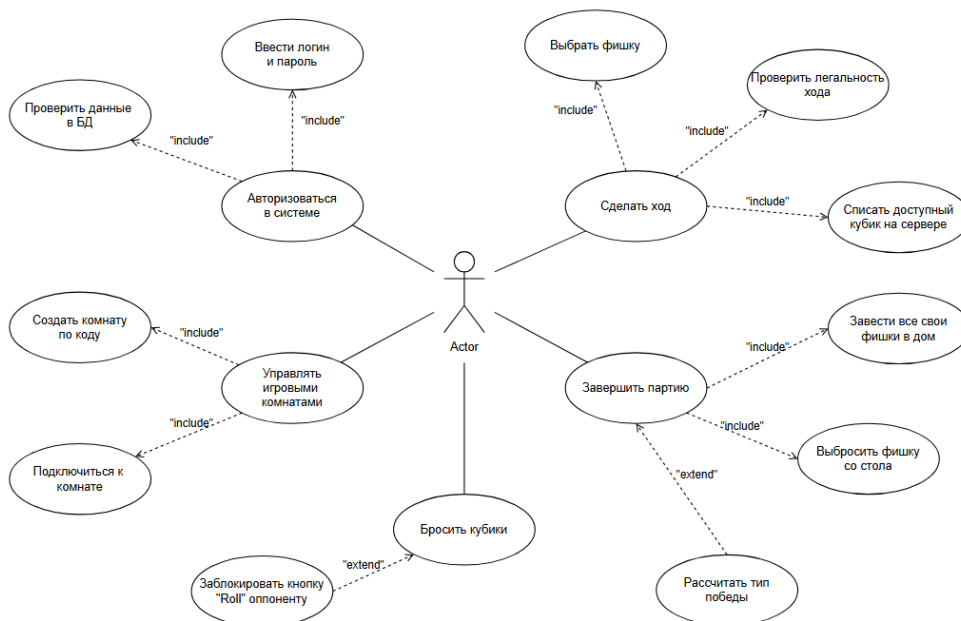


Рисунок 1 - UML-диаграмма вариантов использования

2. Выбор решения

Приложение разработано на языке Java с использованием библиотеки Swing для графического интерфейса, стандартных сетевых сокетов java.net и реляционной СУБД PostgreSQL для хранения данных.

Java — объектно-ориентированный кроссплатформенный язык общего назначения. Обладает развитой стандартной библиотекой, механизмами автоматического управления памятью и встроенной поддержкой многопоточности, что делает его оптимальным выбором для создания отказоустойчивых серверов и сетевых игр.

Swing — легковесная библиотека Java для построения GUI [INDEX]. Предоставляет событийную модель для мгновенной обработки кликов пользователя, а также позволяет реализовывать кастомную графику.

Классы java.net (Sockets) — стандартный сетевой интерфейс Java для доступа к службам стека TCP/IP. Инкапсулируют низкоуровневую передачу байтов, предоставляя потоки ввода-вывода (ObjectOutputStream/ObjectInputStream) для обмена сериализованными объектами.

Клиент-серверное взаимодействие построено по топологии «звезда» на базе гарантированного протокола TCP. Все независимые клиенты подключаются к центральному серверу, который изолирует сессии игроков в потоках и объединяет их в комнаты для проведения матча.

Вся игровая логика и сетевой коннект написаны на языке Java, интерфейс спроектирован на Swing, а централизованное хранение учетных записей пользователей обеспечивает полноценная СУБД PostgreSQL.

3. Описание программы

Данная программа состоит из клиентской части, серверной части, а также части с логикой, выделенной в отдельный пакет. Первым запускается сервер. На остальных компьютерах локальной сети - клиент. Сервер представлен в виде консоли, клиентская часть - оконное приложение.

Программа осуществляет игровой процесс между пользователями в локальной сети. Диаграммы развертывания для классов клиента и сервера показаны на рисунках 2 и 3.

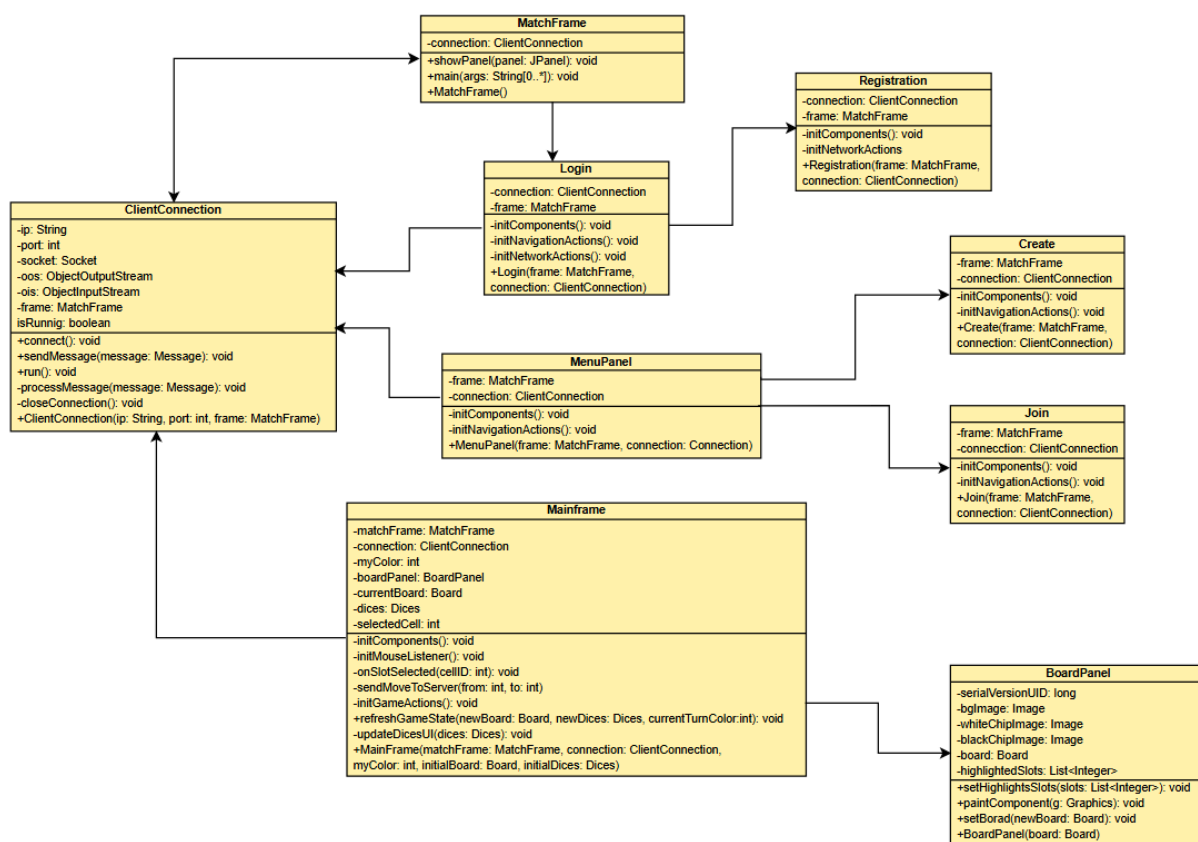


Рисунок 2 - UML-диаграмма классов клиента

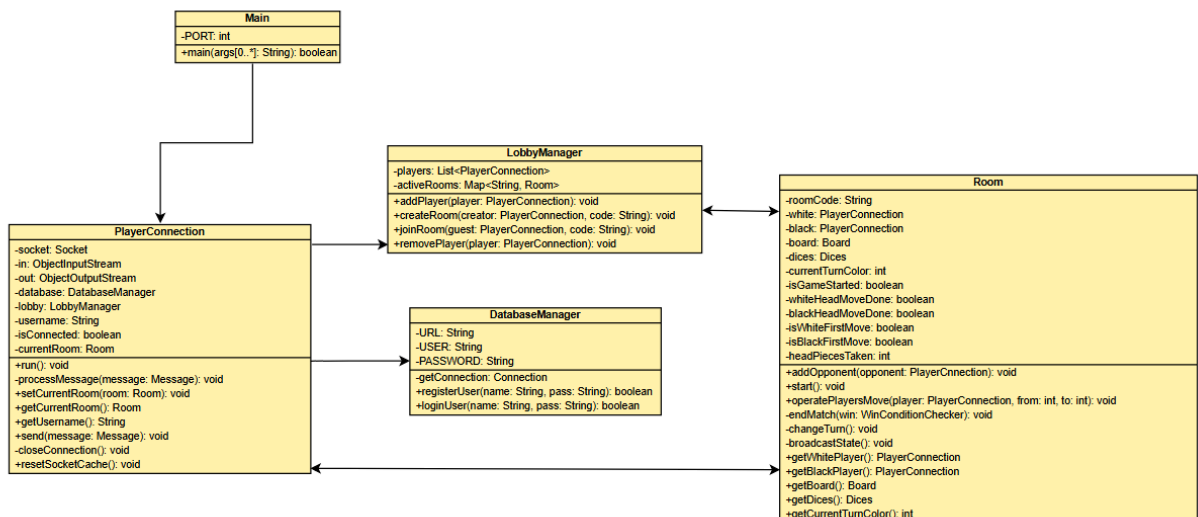


Рисунок 3 - UML-диаграмма классов сервера

3.1 Серверная часть

Main.java – главный файл серверного процесса. Здесь реализуется функция Main. Задается порт и объявляется сокет. Происходит ожидание клиентов для создания с ними связи. В новом потоке создается объект класса PlayerConnection.

PlayerConnection.java – класс, который управляет сетевой сессией конкретного подключенного клиента в отдельном потоке (Runnable). Содержит методы:

processMessage() — главный диспетчер (переключатель) входящих команд, перенаправляющий пакеты Message на регистрацию, авторизацию или в игровую комнату.

send() — выполняет сериализацию и отправку сообщений обратно клиенту с принудительной очисткой кэша сокета.

closeConnection() — безопасно закрывает сетевые потоки и сокет при отключении пользователя.

resetSocketCache() — принудительно очищает внутренний кэш `ObjectOutputStream` через команду `out.reset()`, предотвращая отправку старых (кэшированных) состояний кубиков и доски.

LobbyManager.java — класс, управляющий списком активных комнат и координирующий игроков в общем лобби. Содержит методы:

addPlayer() — регистрирует авторизованного игрока в общем списке лобби, делая его доступным для создания или поиска комнат.

createRoom() — создает новую игровую комнату по уникальному текстовому инвайт-коду, если данный код еще не занят.

joinRoom() — находит созданную комнату по коду, зачисляет туда второго игрока и инициирует запуск матча.

removePlayer() — удаляет игрока из списков лобби при его отключении от сервера, чтобы избежать отправки пакетов на закрытый сокет.

Room.java — класс, отвечающий за игровой сеанс (комнату) между двумя соперниками. Содержит методы:

start() — переводит комнату в игровое состояние, фиксирует цвета пешек и рассылает стартовые пакеты оппонентам.

operatePlayersMove() — обрабатывает игровые действия (бросок кубиков, ход фишкой, сброс с доски), проверяет их легальность и списывает заряды костей.

changeTurn() — переключает очередь хода на противоположный цвет, сбрасывает флаги головы и очищает кубики.

broadcastState() — транслирует текущее состояние доски и кубиков обоим участникам матча.

endMatch() — фиксирует завершение игры, определяет победителя, тип победы и рассылает уведомления.

addOpponent() — записывает подключившегося гостя в поле черного игрока комнаты.

isFull() — проверяет, заняты ли оба игровых места в комнате, сигнализируя о готовности к старту.

DatabaseManager.java — класс, который отвечает за интеграцию сервера с реляционной СУБД PostgreSQL. Содержит методы:

registerUser() — записывает логин и чистую строку пароля нового пользователя в таблицу users, проверяя уникальность имени.

authenticateUser() — сверяет введенные при входе данные с записями в базе данных для подтверждения авторизации.

3.2 Клиентская часть

MatchFrame.java — главное окно приложения (JFrame), выступающее базовым графическим контейнером. При старте открывает TCP-сокеты и запускает фоновую сеть. Содержит метод:

showPanel() — бесшовно очищает окно от старого экрана и монтирует новую панель внутри одного окна, пересчитывая размеры рамок через *pack()*.

ClientConnection.java — фоновый сетевой коннектор клиента (Runnable). Отвечает за сокетное соединение и постоянное прослушивание сети. Содержит методы:

connect() — открывает TCP-сокеты до сервера и инициализирует потоки сериализации *ObjectOutputStream* / *ObjectInputStream*.

sendMessage() — выполняет отправку собранных пакетов *Message* по сети на сервер.

processMessage() — асинхронный переключатель (диспетчер), принимающий ответы сервера (*GAME_START*, *UPDATE_BOARD*, *ERROR*) и обновляющий элементы интерфейса в безопасном потоке *Swing*.

Login.java — панель авторизации (JPanel). Содержит методы:

initComponents() — строит визуальную форму ввода и инициализирует маскированное точками поле JPasswordField.

initNetworkActions() — считывает логин и пароль при нажатии на Sign In и отправляет пакет LOGIN_REQUEST на сервер.

initNavigationActions() — переключает фрейм на экран регистрации при клике на кнопку Sign Up.

Registration.java — панель регистрации (JPanel). Содержит методы:

initNetworkActions() — считывает данные для создания нового аккаунта и шлет запрос REGISTRATION_REQUEST.

initComponents() — строит форму регистрации и кнопку Cancel для возврата пользователя на экран входа.

MenuPanel.java — панель главного меню лобби (JPanel). Содержит методы:

initComponents() — конструирует лобби и верхнее меню Settings с опцией Logout.

initNavigationActions() — перенаправляет игрока на компактные экраны создания комнаты (Create) или подключения по коду (Join).

Create.java — компактная панель создания приватной комнаты (JPanel, размер 400x300). Содержит методы:

initComponents() — центрирует поле ввода инвайт-кода через менеджер компоновки GridBagLayout.

initNavigationActions() — отправляет пакет CREATE_ROOM на сервер, переводит интерфейс в режим ожидания соперника, а по кнопке Cancel возвращает в лобби с восстановлением размеров окна.

Join.java — компактная панель подключения к игре (JPanel, размер 400x300). Содержит методы:

initComponents() — строит поле ввода кода комнаты друга через GridBagLayout.

initNavigationActions() — отправляет запрос JOIN_ROOM на сервер для мгновенного старта матча.

Mainframe.java — основной игровой экран (JPanel). Компонует игровое поле, кубики и кнопки. Содержит методы:

initGameActions() — связывает кнопку Roll и кнопку сброса Bear off с отправкой сетевых команд на сервер.

initMouseListener() — рассчитывает точный номер лунки (от 0 до 23) при клике мышкой по доске.

onSlotSelected() — обрабатывает логику кликов, блокирует нажатия по чужим фишкам и проверяет, брошены ли кубики.

refreshGameState() — принимает обновленную сервером доску и кубики, переключает доступность кнопок и пересчитывает счет на лейблах JLabelWhiteScore / JLabelBlackScore на основе оставшихся шашек.

updateDicesUI() — выводит выпавшие значения кубиков на экран или рисует прочерки при передаче хода.

BoardPanel.java — кастомный графический холст доски (JPanel). Содержит методы:

setBoard() — принимает и обновляет актуальную конфигурацию доски с сервера.

paintComponent() — с помощью контекста Graphics2D со сглаживанием отрисовывает фоновое поле field.png, фишки с прозрачными альфа-каналами и полупрозрачные зеленые маркеры доступных ходов.

3.3 Часть логики

Message.java — базовый класс сетевого пакета (конверта данных), поддерживающий интерфейс `Serializable`. Служит единым стандартом обмена данными между клиентом и сервером. Содержит методы:

getCommand() / *setCommand()* — геттер и сеттер для числового кода команды (`LOGIN_REQUEST`, `UPDATE_BOARD` и т.д.).

getData() / *setData()* — геттер и сеттер для передачи любых объектов (строк, массивов, объектов доски или кубиков).

getErrorMessage() / *setErrorMessage()* — геттер и сеттер для передачи текстовых уведомлений об ошибках.

Cell.java — класс модели одной лунки (игрового пункта) на доске, поддерживающий интерфейс `Serializable`. Содержит методы:

getColor() / *setColor()* — определяет цвет фишек, находящихся в данной лунке (`Cell.WHITE`, `Cell.BLACK` или `Cell.EMPTY`).

getCount() / *setCount()* — возвращает или устанавливает точное количество фишек, стоящих в этой лунке.

isEmpty() — проверяет, пуста ли данная лунка в текущий момент.

removePiece() — уменьшает количество фишек на одну единицу и автоматически сбрасывает цвет лунки на пустой, если фишки закончились.

Board.java — класс модели игрового поля, поддерживающий интерфейс `Serializable`. Представляет собой массив из 24 объектов `Cell`. Содержит методы:

getCell() — возвращает объект конкретной лунки по её индексу (от 0 до 23).

initBoard() — задает начальную расстановку партии (по 15 фишек на «головах» белых и черных — в лунках 0 и 12).

Dices.java — класс модели игральных костей (кубиков), поддерживающий интерфейс `Serializable`. Содержит методы:

roll() — генерирует случайные числа от 1 до 6 для двух кубиков, определяет наличие дубля и рассчитывает количество доступных зарядов (uses).

takeDice() — списывает (уменьшает на единицу) заряд использованного кубика после совершения игроком легального хода.

getAvailableValues() — возвращает список (`List<Integer>`) числовых значений кубиков, которые у игрока остались доступны для ходов в текущем раунде.

clear() — принудительно обнуляет значения и заряды кубиков при передаче очереди хода оппоненту.

MoveValidator.java — класс, отвечающий за строгую проверку ходов на сервере. Содержит методы:

canMove() — проверяет легальность перемещения фишки из одной лунки в другую с учетом цвета игрока, занятости целевой лунки соперником, правил «Головы», «Первого хода» и финишного сброса (`to == -1`).

move() — физически перемещает фишку на доске (уменьшает счетчик в стартовой лунке и увеличивает в целевой).

isHeadPosition() — определяет, является ли выбранная лунка «головой» для текущего цвета.

isDiceValueMatching() — проверяет, соответствует ли физическое расстояние между лунками номиналу кубика с учетом направления движения.

MoveCalculator.java — вспомогательный класс для расчета возможных вариантов ходов. Содержит методы:

getPossibleMoves() — сканирует доску и возвращает список доступных лунок (индексов), куда выбранная фишка может наступить текущими кубиками.

hasAnyValidMoves() — проверяет, есть ли у игрока хотя бы один легальный ход оставшимися кубиками.

WinConditionChecker.java — класс, анализирующий состояние доски на предмет завершения матча. Содержит методы:

isGameOver() — сканирует доску и определяет, выбросил ли кто-то из игроков все свои 15 фишек за поле.

getWinnerColor() — возвращает цвет игрока, который первым успешно завершил сброс фишек.

calculateVictoryType() — анализирует положение фишек проигравшего и определяет тип победы по правилам нард (Ойн или Марс) для начисления очков.

Диаграмма классов

4. Описание способа организации пользовательского интерфейса

Графический пользовательский интерфейс (GUI) разработан на базе стандартной кроссплатформенной библиотеки `javax.swing`. Интерфейс построен по принципу однооконного приложения (Single Frame Architecture), где главным контейнером выступает класс `MatchFrame` (extends `JFrame`).

Вместо открытия множества независимых окон навигация реализована через бесшовное переключение графических панелей класса `JPanel` внутри единого родительского окна. Переключение экранов выполняет метод `showPanel(JPanel panel)`, который очищает контейнер, монтирует новую панель и вызывает метод `frame.pack()`. Метод `pack()` автоматически пересчитывает геометрические размеры рамок ОС под внутреннее содержимое, предотвращая урезание элементов интерфейса. Вся структура интерфейса разделена на три логические стадии: авторизация и регистрация (`Login`, `Registration`); игровое лобби (`MenuPanel`, `Create`, `Join`); игровой процесс (`Mainframe`, `BoardPanel`).

Визуальной основой игрового экрана является кастомный холст `BoardPanel`, в котором переопределен метод `paintComponent(Graphics g)`. Отрисовка поля выстроена по принципу послойного наложения графики с использованием контекста `Graphics2D` и фильтров сглаживания (`Antialiasing`).

Интерфейс полностью синхронизирован с фоновым сетевым потоком `ClientConnection`. Обновление графики (перемещение фишек, изменение цифр счета и кубиков, блокировка кнопок) выполняется строго внутри системного потока `SwingUtilities.invokeLater()`. Это защищает интерфейс от зависаний при асинхронном получении пакетов `UPDATE_BOARD` от сервера и обеспечивает мгновенный отклик приложения.

5. Описание результатов работы программы

Для начала необходимо запустить сервер (рис. 4).

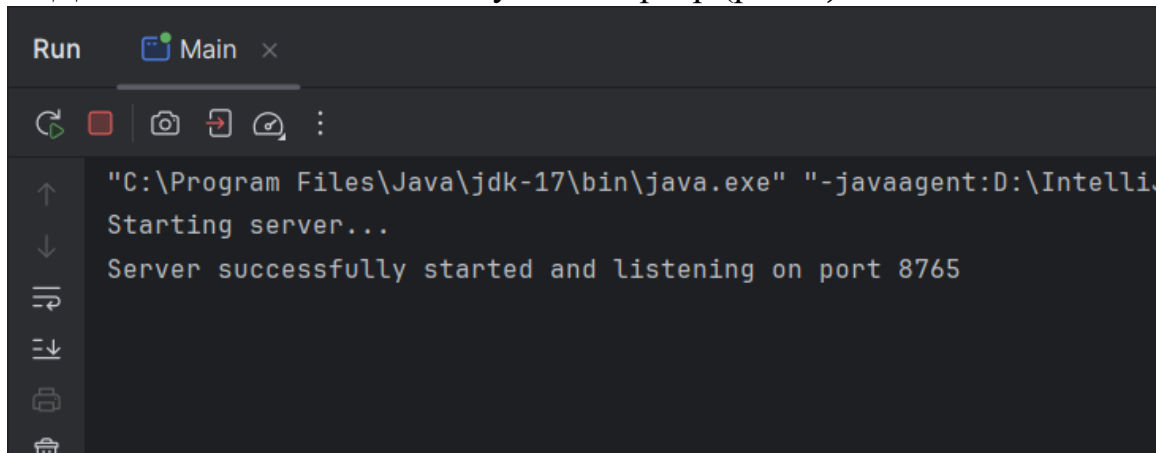


Рисунок 4 - Запуск сервера

Далее запускаем клиент и у нас открывается окно входа (рис. 5).

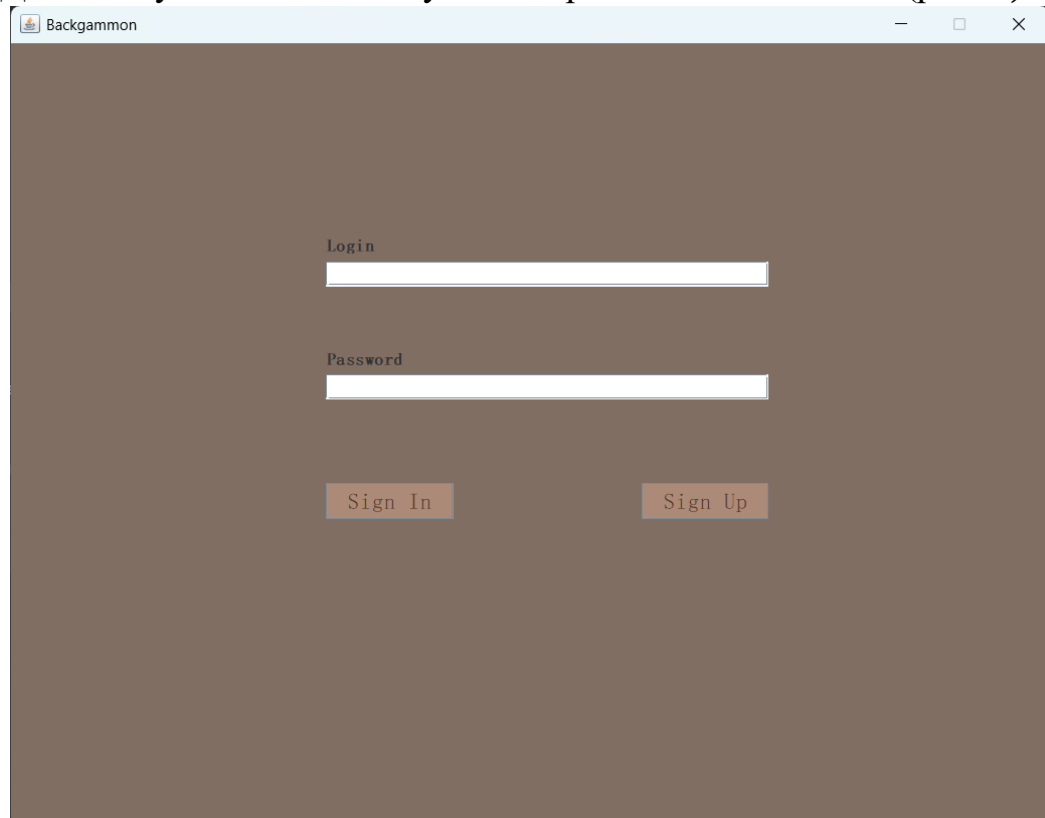


Рисунок 5 - Окно входа в аккаунт

Если необходимо создать аккаунт, то стоит в окне входа нажать кнопку «Sign Up», после чего в открывшемся окне придумать и ввести логин и пароль. По окончании ввода нажать на кнопку «Sign Up», а затем на кнопку «Cancel», чтобы выйти в окно входа (рис. 6).

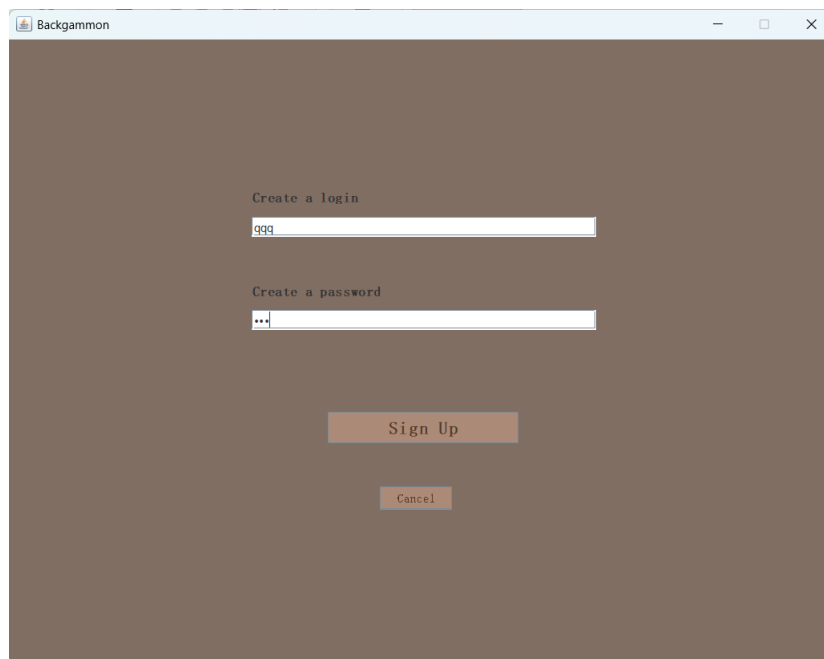
The screenshot shows a window titled "Backgammon" with a dark brown background. It contains two text input fields. The first is labeled "Create a login" and contains the text "qqq". The second is labeled "Create a password" and contains three dots "..." indicating a masked password. Below the fields are two buttons: "Sign Up" and "Cancel".

Рисунок 6 - Окно регистрации

Вводим логин и пароль, после чего стоит нажать на кнопку «Sign In», чтобы войти в меню (рис. 7).

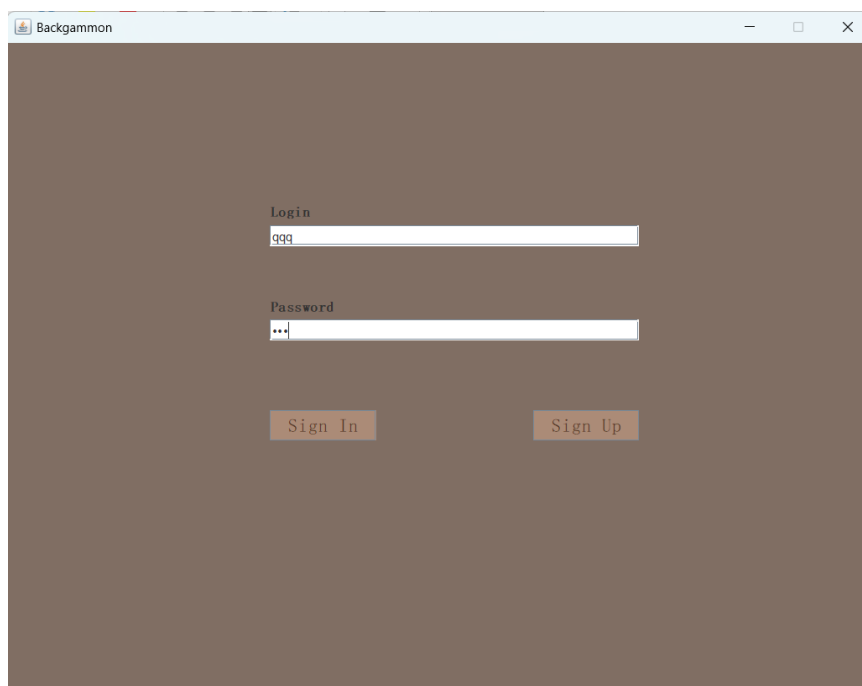
The screenshot shows a window titled "Backgammon" with a dark brown background. It contains two text input fields. The first is labeled "Login" and contains the text "qqq". The second is labeled "Password" and contains three dots "..." indicating a masked password. Below the fields are two buttons: "Sign In" and "Sign Up".

Рисунок 7 - Ввод логина и пароля

При входе в меню нас встречают две кнопки: «Create the game» - для создания игры, и «Join the game» - для подключения к игре (рис. 8).

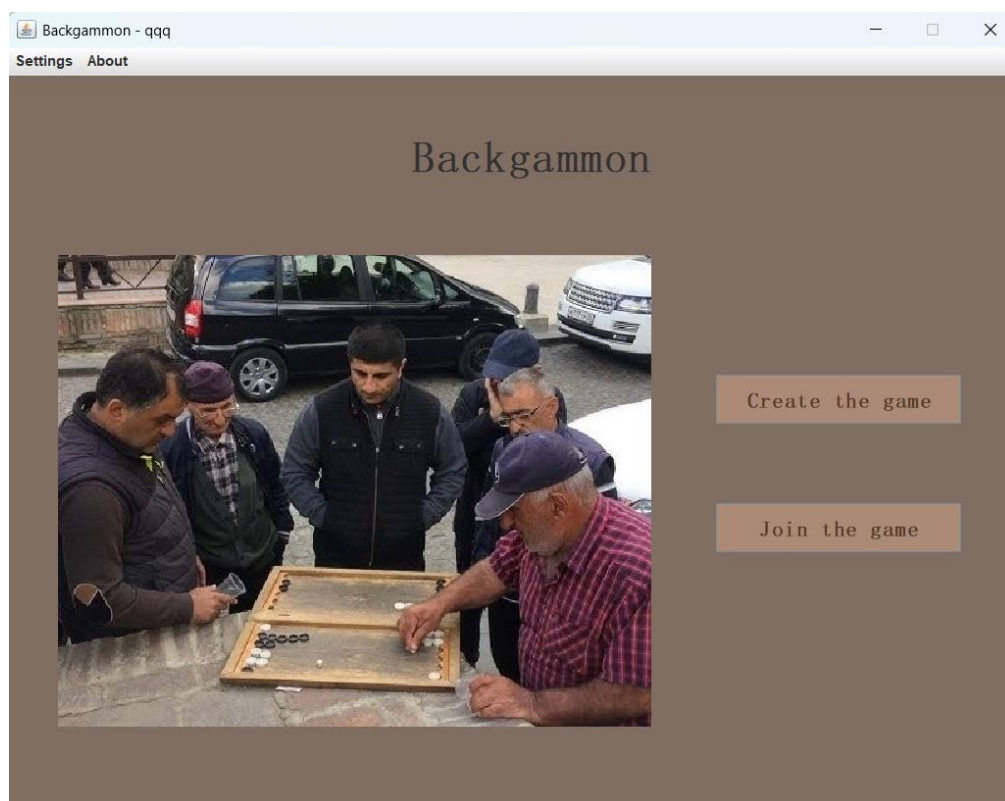


Рисунок 8 - Окно меню

По нажатию кнопки «Create the game» открывается окно, в котором стоит придумать код для игры, после чего нажать на «Create» (рис. 9)



Рисунок 9 - Ввод кода для создания игры

По нажатию кнопки «Create», всплывает окно, сообщающее об ожидании соперника (рис. 10).

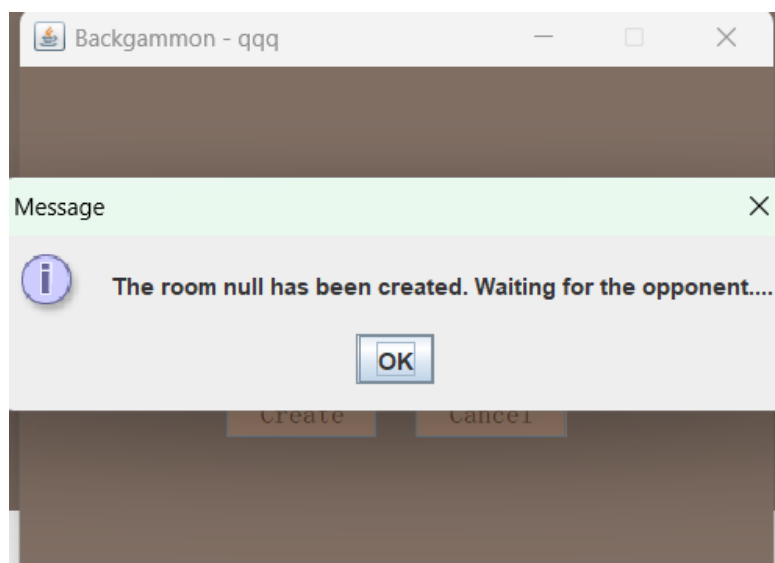


Рисунок 10 - Уведомление об ожидании соперника

Оппонент после входа в аккаунт должен нажать кнопку «Join the game», чтобы подключиться к игре (рис. 11).

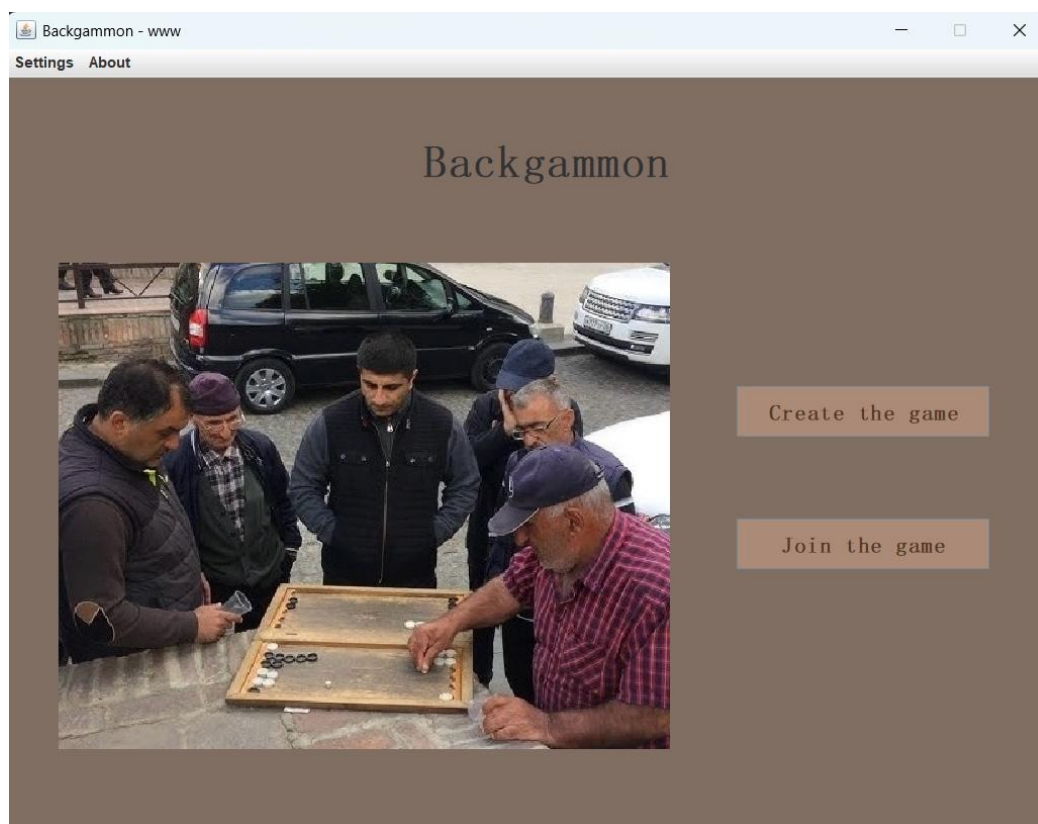


Рисунок 11 - Окно меню соперника

Далее необходимо ввести код игры и нажать на «Join» (рис. 12).



Рисунок 12 - Ввод кода для подключения к матчу

После того, как соперник нажал на «Join», оба игрока попадают в игровую комнату (рис. 13, 14).

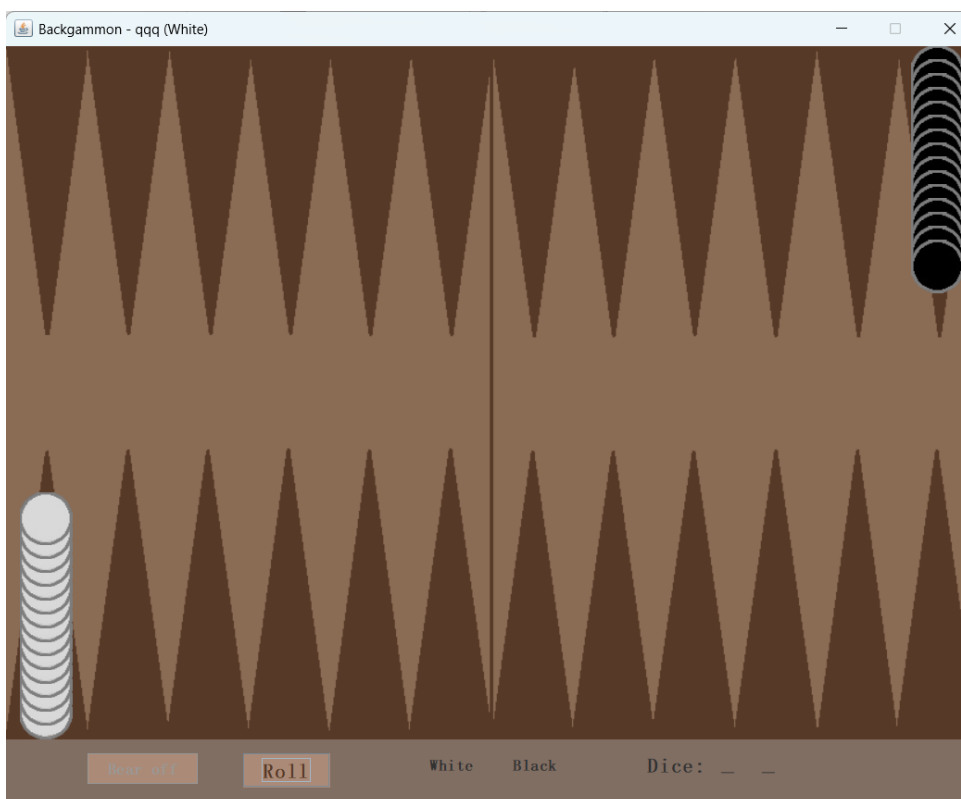


Рисунок 13 - Игровое поле белых

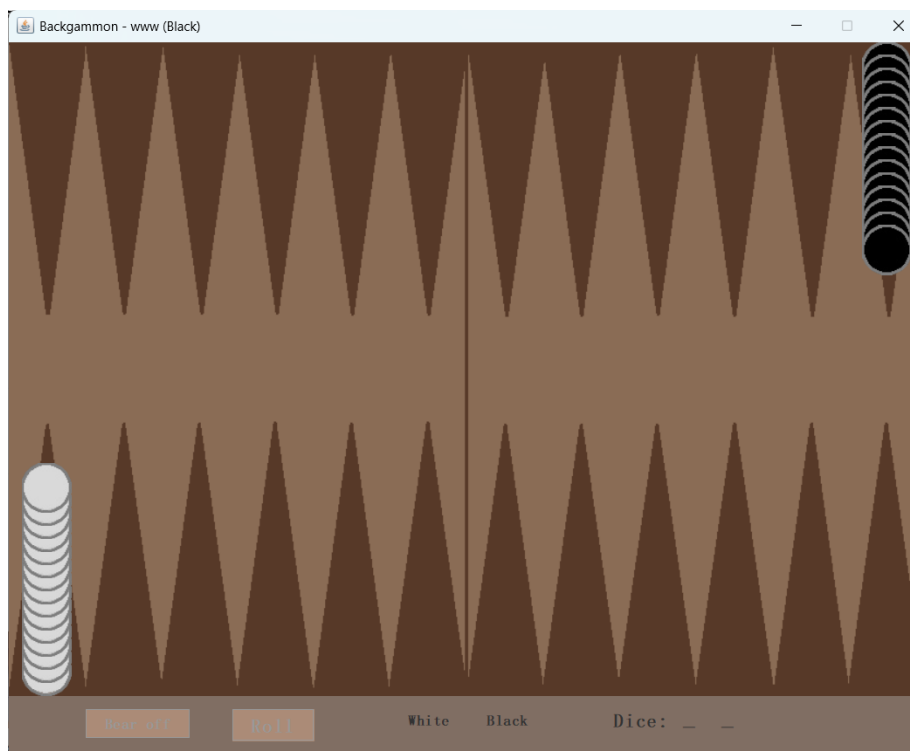


Рисунок 14 - Игровое поле черных

У каждого игрока есть кнопка «Roll», чтобы бросить кубики. После того, как кубики брошены, можно совершить два или четыре хода (в зависимости от выпавших кубиков), равные значению одного кубика (рис. 15, 16).

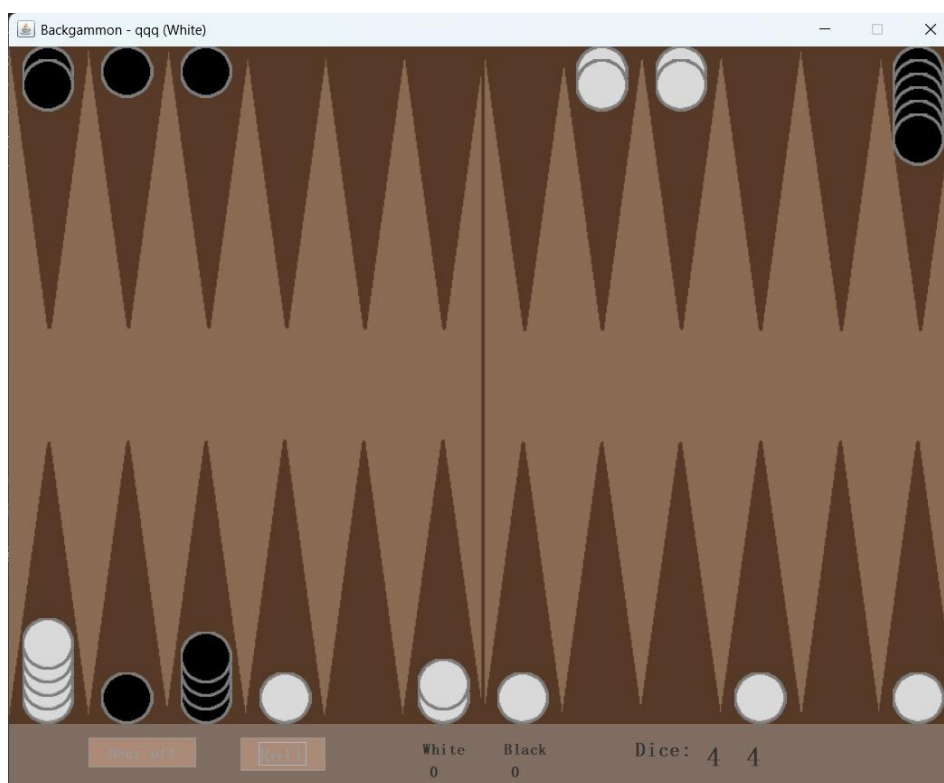


Рисунок 15 - Процесс игры за белых

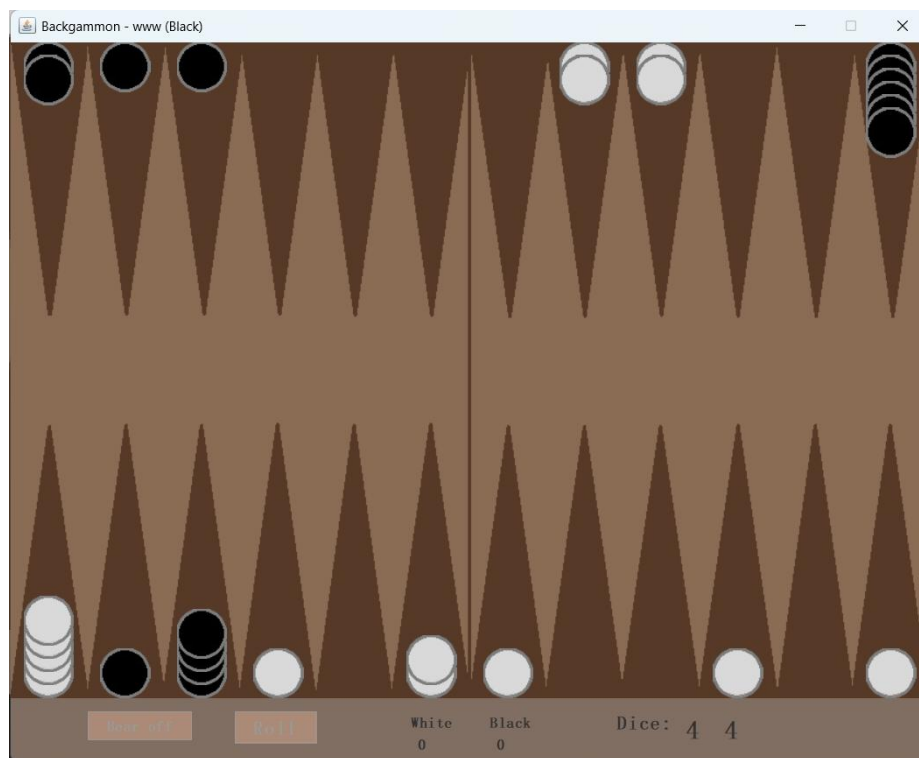


Рисунок 16 - Процесс игры за черных

После того, как все фишки будут находиться в доме, станет доступна еще одна кнопка – «Bear off». Она служит для того, чтобы выбрасывать фишки с доски (рис. 17).

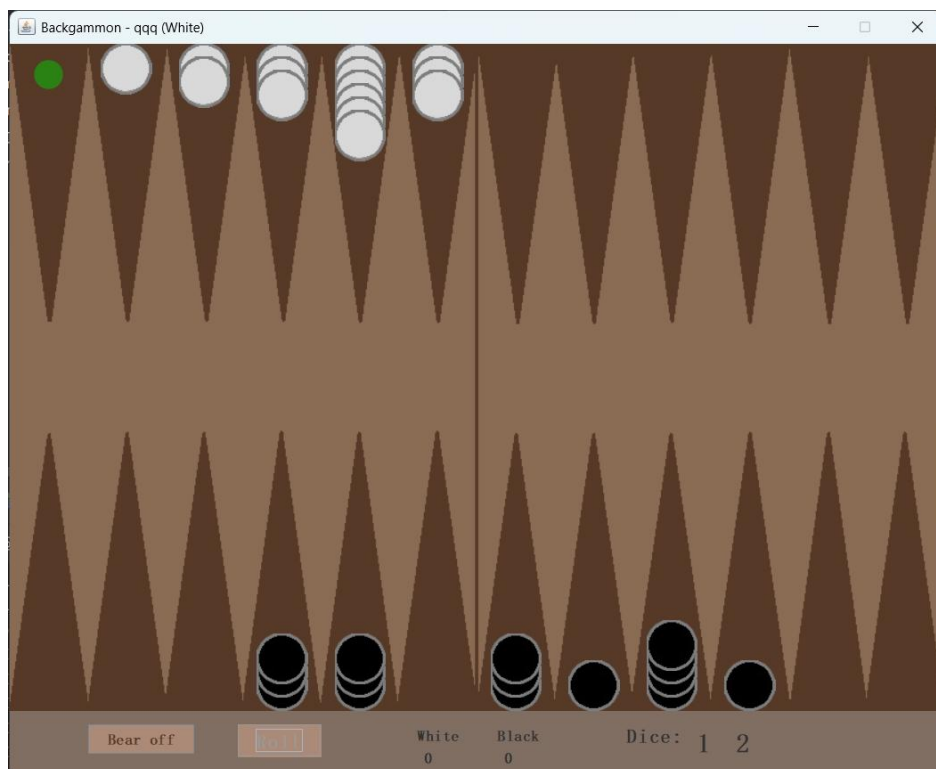


Рисунок 17 - Стадия выброса фишек со стола

После того, как один из игроков выбросит все свои фишки со стола, игра останавливается, и победа присуждается этому игроку. Далее всплывает информационное окно с именем победителя и с кнопкой «Ок», по нажатию которой, игрок переходит в меню (рис 18, 19). Диаграммы последовательности и развертывания представлены в приложении Б.

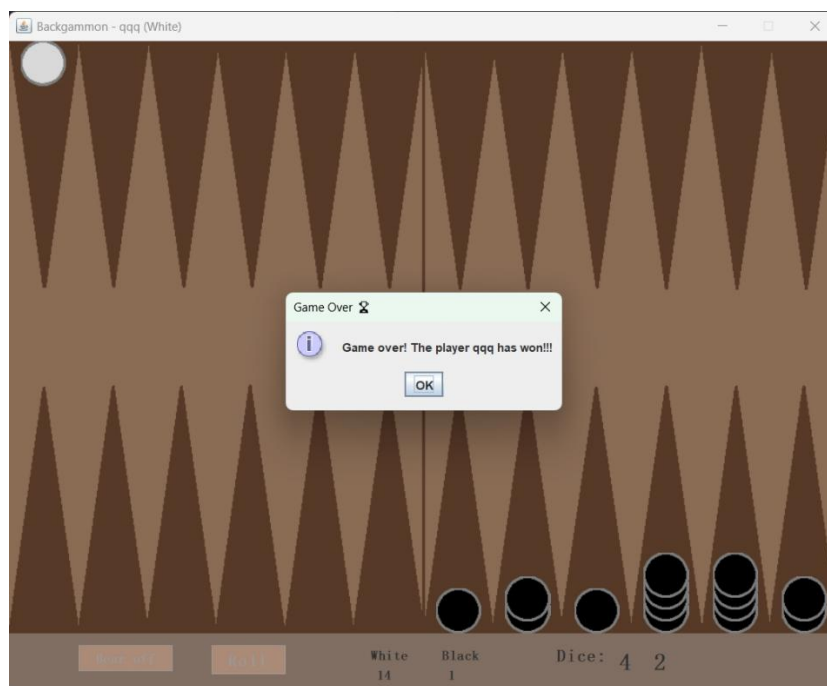


Рисунок 18 - Конец игры для белых

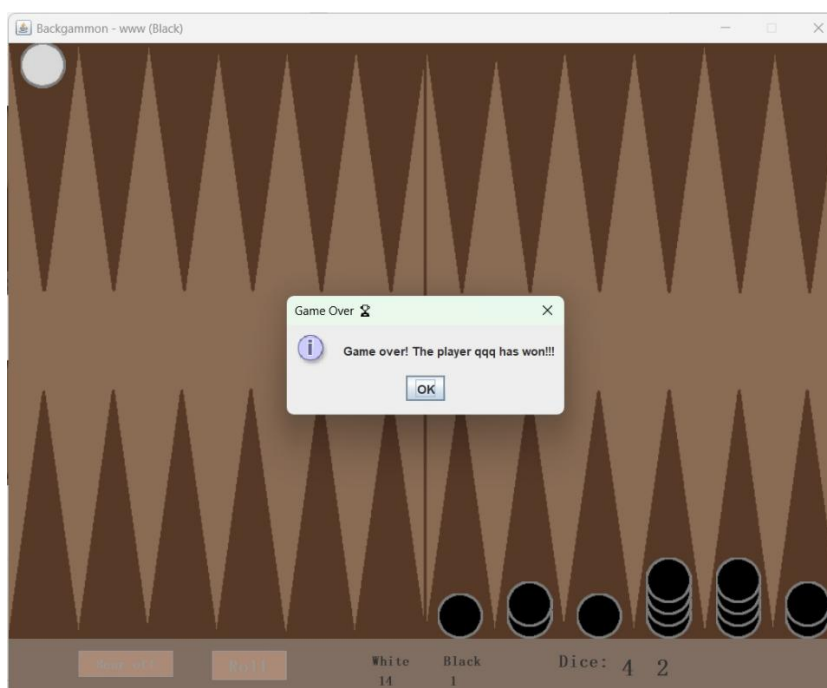


Рисунок 19 - Конец игры для черных

Заключение

При выполнении данной курсовой работы были получены навыки разработки программ на языке высокого уровня. Были освоены приемы создания графического интерфейса. Усвоены механизмы реализации меню. Были закреплены знания и приобретены практические навыки разработки многомодульных приложений на языке Java. Изучены основные возможности среды программирования IntelliJ IDEA.

В ходе данной курсовой работы была разработана классическая игра «Длинные нарды».

В дальнейшем это приложение можно будет улучшить, добавив игру против бота, игру с соперником на одном окне (на одном компьютере). Также можно будет оптимизировать код для рационального использования графических ресурсов и оперативной памяти.

Список литературы

1. Берд, Барри Java для чайников / Барри Берд. - М.: Диалектика / Вильямс, 2013. - 521 с.
2. Дубаков А.А. Сетевое программирование: учебное пособие / А.А. Дубаков – СПб: НИУ ИТМО, 2013. – 248 с.
3. Java. Полное руководство, 12-е изд. : Пер. с англ. - СПб. "Диалектика"; 2023. - 1344 с.: ил. - Парал. тит. англ.

Приложение А. Листинг программы

Листинг клиента

Файл ClientConnection.java

```
package ru.psu.coursework.client.network;

import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.additional.models.Board;
import ru.psu.coursework.additional.models.Cell;
import ru.psu.coursework.additional.models.Dices;
import ru.psu.coursework.client.ui.Login;
import ru.psu.coursework.client.ui.Mainframe;
import ru.psu.coursework.client.ui.MatchFrame;
import ru.psu.coursework.client.ui.MenuPanel;

import javax.swing.*;
import java.io.*;
import java.net.*;

public class ClientConnection implements Runnable {
    private String ip;
    private int port;
    private Socket socket;
    private ObjectOutputStream oos;
    private ObjectInputStream ois;
    private final MatchFrame frame;
    boolean isRunning = false;

    public ClientConnection(String ip, int port, MatchFrame frame) {
        this.ip = ip;
        this.port = port;
        this.frame = frame;
    }

    public void connect() throws IOException {
        this.socket = new Socket(ip, port);
        this.oos = new ObjectOutputStream(socket.getOutputStream());
        this.ois = new ObjectInputStream(socket.getInputStream());
        this.isRunning = true;
    }
}
```

```

    }

    public synchronized void sendMessage(Message message) throws IOException {
        if (oos != null) {
            oos.writeObject(message);
            oos.flush();
        }
    }

    @Override
    public void run() {
        try {
            while (isRunning) {
                Message incomingMessage = (Message) ois.readObject();

                processMessage(incomingMessage);
            }
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Connection to the server was lost: " +
e.getMessage());
        } finally {
            closeConnection();
        }
    }

    //ответ сервера
    private void processMessage(Message message) {
        SwingUtilities.invokeLater(() -> {
            if (message.getCommand() == Commands.AUTHENTICATION_SUCCESS) {
                System.out.println("AUTH_SUCCESS received, data: " +
message.getData());
                System.out.println("Data class: " + (message.getData() != null
? message.getData().getClass() : "null"));

                String name = (String) message.getData();

                //final String finalName = name;
                System.out.println("Extracted name: '" + name + "'");

                SwingUtilities.invokeLater(() -> {
                    frame.showPanel(new MenuPanel(frame, this));
                });
            }
        });
    }

```

```

        frame.setTitle("Backgammon - " + name);
    });

    } else if (message.getCommand() == Commands.CREATE_ROOM) {
        String createdCode = (String) message.getData();

        JOptionPane.showMessageDialog(frame, "The room " + createdCode
+ " has been created. Waiting for the opponent....");

    } else if (message.getCommand() == Commands.GAME_START) {
        try {
            Object[] startData = (Object[]) message.getData();

            int myColor = Integer.parseInt(startData[0].toString());
            Dices initialDices = (Dices) startData[1];
            Board initialBoard = (Board) startData[2];

            javax.swing.SwingUtilities.invokeLater(() -> {
                frame.showPanel(new Mainframe(frame, this, myColor,
initialBoard, initialDices));

                String colorText = (myColor == Cell.WHITE) ? " (White)"
: " (Black)";

                frame.setTitle(frame.getTitle() + colorText);

                frame.pack();
                frame.setLocationRelativeTo(null);
            });

        } catch (Exception e) {
            System.err.println("Ошибка на клиенте при старте игры: " +
e.getMessage());
            e.printStackTrace();
        }

    } else if (message.getCommand() == Commands.UPDATE_BOARD) {
        System.out.println("CLIENT: Received UPDATE_BOARD packet from
server!");

        try {
            Object[] gameData = (Object[]) message.getData();
            Board updatedBoard = (Board) gameData[0];

```

```

        Dices updatedDices = (Dices) gameData[1];
        int currentTurnColor = (int) gameData[2];

        javax.swing.SwingUtilities.invokeLater(() -> {
            Mainframe activeGameWindow = null;
            for (java.awt.Component comp :
frame.getContentPane().getComponents()) {
                if (comp instanceof Mainframe) {
                    activeGameWindow = (Mainframe) comp;
                    break;
                }
            }

            if (activeGameWindow != null) {
                System.out.println("CLIENT: Game window found,
calling refreshGameState...");
                activeGameWindow.refreshGameState(updatedBoard,
updatedDices, currentTurnColor);
            } else {
                System.err.println("CLIENT ERROR: Mainframe panel
not found among active window components!");
            }

        });
    } catch (Exception e) {
        System.err.println("Error unpacking UPDATE_BOARD on the
client: " + e.getMessage());
        e.printStackTrace();
    }

    } else if (message.getCommand() == Commands.ERROR) {
        //JOptionPane.showMessageDialog(frame,
message.getErrorMessage(), "Server Error!!!", JOptionPane.ERROR_MESSAGE);
        String errorMsg = message.getErrorMessage();

        String windowTitle = "Server Error!!!";
        int messageType = JOptionPane.ERROR_MESSAGE;

        if (errorMsg != null && errorMsg.contains("Game over")) {
            windowTitle = "Game Over 🏆";
            messageType = JOptionPane.INFORMATION_MESSAGE;
        }
    }
}

```

```

        JOptionPane.showMessageDialog(frame, errorMsg, windowTitle,
messageType);

        if (errorMsg != null && errorMsg.contains("Game over")) {
            System.out.println("CLIENT: Game over. Returning user to
the main lobby menu");

            javax.swing.SwingUtilities.invokeLater(() -> {
                frame.setSize(840, 660);
                frame.setLocationRelativeTo(null);
                frame.showPanel(new MenuPanel(frame, this));
            });
        }

    } else if (message.getCommand() == Commands.REGISTRATION_REQUEST)
{
        String result = (String) message.getData();

        if ("Success".equals(result) || message.getErrorMessage() ==
null) {
            JOptionPane.showMessageDialog(frame,
                "Регистрация прошла успешно! Теперь вы можете
войти.",
                "Успех", JOptionPane.INFORMATION_MESSAGE);

            frame.showPanel(new Login(frame, this));
        }
    }
});
}

private void closeConnection() {
    isRunning = false;

    try {
        if (ois != null) ois.close();
        if (oos != null) oos.close();
        if (socket != null && !socket.isClosed()) socket.close();
    } catch (IOException e) {

```



```

        e.printStackTrace();
    }
}

```

Файл BoardPanel.java

```

package ru.psu.coursework.client.ui;

import ru.psu.coursework.additional.models.Board;
import ru.psu.coursework.additional.models.Cell;

import javax.swing.*;
import java.awt.*;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class BoardPanel extends JPanel implements Serializable {
    private static final long serialVersionUID = 1L;

    private Image bgImage;
    private Image whiteChipImage;
    private Image blackChipImage;

    private Board board;
    private List<Integer> highlightedSlots = new ArrayList<>();

    public BoardPanel(Board board) {
        this.board = board;

        bgImage = new
        ImageIcon(getClass().getResource("/ru/psu/coursework/other/field.png")).getIm
        age();

        whiteChipImage = new
        ImageIcon(getClass().getResource("/ru/psu/coursework/other/white1.png")).getI
        mage();

        blackChipImage = new
        ImageIcon(getClass().getResource("/ru/psu/coursework/other/black1.png")).getI
        mage();

        setPreferredSize(new Dimension(840, 600));
    }
}

```

```

    }

    //подсветка нужных мест
    public void setHighlightedSlots(List<Integer> slots) {
        this.highlightedSlots = slots;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_INTERPOLATION,
RenderingHints.VALUE_INTERPOLATION_BILINEAR);
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        if (bgImage != null) {
            g2d.drawImage(bgImage, 0, 0, 840, 600, this);
        }

        if (board == null) return;

        int chipSize = 46;
        int overlapStep = 12;//расстояние наложения
        int centerOffset = (70 - chipSize) / 2;

        for (int i = 0; i < 24; i++) {
            Cell cell = board.getCell(i);
            if (cell == null || cell.isEmpty()) continue;

            Image currentChipImg = (cell.getColor() == Cell.WHITE) ?
whiteChipImage : blackChipImage;
            if (currentChipImg == null) continue;

            int baseX = (i <= 11) ? (i * 70) : ((23 - i) * 70);
            int finalX = baseX + centerOffset;

            for (int k = 0; k < cell.getCount(); k++) {
                int finalY;
                if (i <= 11) {
                    finalY = 600 - chipSize - (k * overlapStep);

```

```

        } else {
            finalY = 0 + (k * overlapStep);
        }

        g2d.drawImage(currentChipImg,    finalX,    finalY,    chipSize,
chipSize, this);
    }
}

if (highlightedSlots != null && !highlightedSlots.isEmpty()) {
    g2d = (Graphics2D) g;
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

    g2d.setColor(new Color(0, 200, 0, 130));

    for (int cellId : highlightedSlots) {
        if (cellId < 0 || cellId >= 24) continue;

        int baseX = (cellId <= 11) ? (cellId * 70) : ((23 - cellId) *
70);

        int markerSize = 26;
        int x = baseX + (70 - markerSize) / 2;
        int y = (cellId <= 11) ? (600 - markerSize - 15) : (0 + 15);

        //кружок
        g2d.fillOval(x, y, markerSize, markerSize);

        g2d.setColor(new Color(0, 200, 0, 130));
    }
}

}

public void setBoard(Board newBoard) {
    this.board = newBoard;
}

}

```

Файл Create.java

```
package ru.psu.coursework.client.ui;
```

```

import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.client.network.ClientConnection;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.IOException;

public class Create extends javax.swing.JPanel {

    private final MatchFrame frame;
    private final ClientConnection connection;
    private JTextField jTextField1;
    private JButton jButtonCreate;
    private JButton jButtonCancel;

    public Create(MatchFrame frame, ClientConnection connection) {
        this.frame = frame;
        this.connection = connection;
        initComponents();
        initNavigationActions();
    }

    private void initComponents() {
        setPreferredSize(new Dimension(400, 300));
        setBackground(new java.awt.Color(128, 110, 99));

        setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(10, 20, 10, 20); // Отступы между элементами

        JLabel jLabel1 = new JLabel("Create code for the game:");
        jLabel1.setFont(new java.awt.Font("SimSun-ExtB", Font.BOLD, 14));
        jLabel1.setForeground(Color.BLACK);
        gbc.gridx = 0;
        gbc.gridy = 0;
        gbc.gridwidth = 2;
        add(jLabel1, gbc);
    }

```

```

jTextField1 = new JTextField();
jTextField1.setPreferredSize(new Dimension(200, 30));
gbc.gridy = 1;
add(jTextField1, gbc);

JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20,
0));

buttonPanel.setOpaque(false);

jButtonCreate = new JButton("Create");
jButtonCreate.setBackground(new java.awt.Color(171, 139, 119));
jButtonCreate.setFont(new java.awt.Font("SimSun-ExtB", Font.PLAIN,
14));
jButtonCreate.setForeground(new java.awt.Color(86, 57, 39));

jButtonCancel = new JButton("Cancel");
jButtonCancel.setBackground(new java.awt.Color(171, 139, 119));
jButtonCancel.setFont(new java.awt.Font("SimSun-ExtB", Font.PLAIN,
14));
jButtonCancel.setForeground(new java.awt.Color(86, 57, 39));

buttonPanel.add(jButtonCreate);
buttonPanel.add(jButtonCancel);

gbc.gridy = 2;
gbc.insets = new Insets(20, 20, 10, 20); // Чуть увеличиваем отступ
сверху для кнопок
add(buttonPanel, gbc);
}

private void initNavigationActions() {
    jButtonCancel.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            System.out.println("Creation canceled. Returning to main
menu.");

            frame.setSize(840, 660);
            frame.setLocationRelativeTo(null);
            frame.showPanel(new MenuPanel(frame, connection));
        }
    });
}

```

```

jButtonCreate.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String roomCode = jTextField1.getText().trim();
        if (roomCode.isEmpty()) {
            JOptionPane.showMessageDialog(Create.this,
                "Please enter a room code!", "Warning",
JOptionPane.WARNING_MESSAGE);
            return;
        }

        JOptionPane.showMessageDialog(frame,
            "Waiting for an opponent...",
            "Room " + roomCode,
            JOptionPane.INFORMATION_MESSAGE);

        Message createMsg = new Message(Commands.CREATE_ROOM,
roomCode);

        createMsg.setCommand(Commands.CREATE_ROOM);
        createMsg.setData(roomCode);

        System.out.println("Sending a request to CREATE a room with
code: " + roomCode);
        try {
            connection.sendMessage(createMsg);
        } catch (IOException ex) {
            throw new RuntimeException(ex);
        }
    }
});
}
}

```

Файл Join.java

```

package ru.psu.coursework.client.ui;

import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.client.network.ClientConnection;
import javax.swing.*;
import java.awt.*;

```

```

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.IOException;

public class Join extends javax.swing.JPanel {

    private final MatchFrame frame;
    private final ClientConnection connection;
    private JTextField jTextField2;
    private JButton jButtonJoin;
    private JButton jButtonCancel;

    public Join(MatchFrame frame, ClientConnection connection) {
        this.frame = frame;
        this.connection = connection;
        initComponents();
        initNavigationActions();
    }

    private void initComponents() {
        setPreferredSize(new Dimension(400, 300));
        setBackground(new java.awt.Color(128, 110, 99));

        setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(10, 20, 10, 20);

        JLabel jLabel1 = new JLabel("Enter code for the game:");
        jLabel1.setFont(new java.awt.Font("SimSun-ExtB", Font.BOLD, 14));
        jLabel1.setForeground(Color.BLACK);
        gbc.gridx = 0;
        gbc.gridy = 0;
        gbc.gridwidth = 2;
        add(jLabel1, gbc);

        jTextField2 = new JTextField();
        jTextField2.setPreferredSize(new Dimension(200, 30));
        gbc.gridy = 1;
        add(jTextField2, gbc);
    }

```

```

JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20,
0));

buttonPanel.setOpaque(false);

jButtonJoin = new JButton("Join");
jButtonJoin.setBackground(new java.awt.Color(171, 139, 119));
jButtonJoin.setFont(new java.awt.Font("SimSun-ExtB", Font.PLAIN, 14));
jButtonJoin.setForeground(new java.awt.Color(86, 57, 39));

jButtonCancel = new JButton("Cancel");
jButtonCancel.setBackground(new java.awt.Color(171, 139, 119));
jButtonCancel.setFont(new java.awt.Font("SimSun-ExtB", Font.PLAIN,
14));
jButtonCancel.setForeground(new java.awt.Color(86, 57, 39));

buttonPanel.add(jButtonJoin);
buttonPanel.add(jButtonCancel);

gbc.gridy = 2;
gbc.insets = new Insets(20, 20, 10, 20);
add(buttonPanel, gbc);
}

private void initNavigationActions() {
    jButtonCancel.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            System.out.println("Joining canceled. Returning to main
menu.");

            frame.setSize(840, 660);
            frame.setLocationRelativeTo(null);
            frame.showPanel(new MenuPanel(frame, connection));
        }
    });

    jButtonJoin.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            String roomCode = jTextField2.getText().trim();
            if (roomCode.isEmpty()) {
                JOptionPane.showMessageDialog(Join.this,

```



```

        "Please enter a room code to join!", "Warning",
JOptionPane.WARNING_MESSAGE);
        return;
    }

    Message joinMsg = new Message(Commands.JOIN_ROOM, roomCode);
    joinMsg.setCommand(Commands.JOIN_ROOM);
    joinMsg.setData(roomCode);

    System.out.println("Sending a request to JOIN room with code:
" + roomCode);
    try {
        connection.sendMessage(joinMsg);
    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
}

});
}
}

```

Файл Login.java

```

package ru.psu.coursework.client.ui;

import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.client.network.ClientConnection;

import javax.swing.*;
import java.io.IOException;

public class Login extends javax.swing.JPanel {

    /**
     * Creates new form Login
     */

    private final ClientConnection connection;

    public Login(MatchFrame frame, ClientConnection connection) {
        this.frame = frame;
        this.connection = connection;
    }

```

```

        initComponents();
        initNavigationActions();
        initNetworkActions();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-
BEGIN: initComponents
    private void initComponents() {

        jPanel1 = new javax.swing.JPanel();
        jTextField1 = new javax.swing.JTextField();
        jTextField2 = new javax.swing.JPasswordField();
        jButton1 = new javax.swing.JButton();
        jButton2 = new javax.swing.JButton();
        jLabel1 = new javax.swing.JLabel();
        jLabel2 = new javax.swing.JLabel();

        //setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

        jPanel1.setBackground(new java.awt.Color(128, 110, 99));

        jTextField1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                jTextField1ActionPerformed(evt);
            }
        });

        jButton1.setBackground(new java.awt.Color(171, 139, 119));
        jButton1.setFont(new java.awt.Font("SimSun-ExtB", 0, 18)); // NOI18N
        jButton1.setForeground(new java.awt.Color(86, 57, 39));
        jButton1.setText("Sign In");

        jButton2.setBackground(new java.awt.Color(171, 139, 119));
        jButton2.setFont(new java.awt.Font("SimSun-ExtB", 0, 18)); // NOI18N
        jButton2.setForeground(new java.awt.Color(86, 57, 39));

```

```

jButton2.setText("Sign Up");

jLabel1.setFont(new java.awt.Font("SimSun-ExtB", 1, 14)); // NOI18N
jLabel1.setText("Login");

jLabel2.setFont(new java.awt.Font("SimSun-ExtB", 1, 14)); // NOI18N
jLabel2.setText("Password");

    javax.swing.GroupLayout jPanel1Layout = new
javax.swing.GroupLayout(jPanel1);
    jPanel1.setLayout(jPanel1Layout);
    jPanel1Layout.setHorizontalGroup(

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel1Layout.createSequentialGroup()
            .addGap(251, 251, 251)

        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)

        .addGroup(jPanel1Layout.createSequentialGroup()

        .addComponent(jButton1)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 150,
Short.MAX_VALUE)

        .addComponent(jButton2))
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addComponent(jTextField1)
                .addComponent(jTextField2)
                .addComponent(jLabel2)
                .addComponent(jLabel1)
                .addGap(245, Short.MAX_VALUE))
            .addContainerGap());
    jPanel1Layout.setVerticalGroup(

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel1Layout.createSequentialGroup()
            .addGap(154, 154, 154)

```

```

        .addComponent(jLabel1)

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(jTextField1,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(50, 50, 50)
        .addComponent(jLabel2)

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(jTextField2,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(66, 66, 66)

    .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
    .BASELINE)

        .addComponent(jButton1)
        .addComponent(jButton2))
    .addContainerGap(280, Short.MAX_VALUE)

    );

//          javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
//      getContentPane().setLayout(layout);
//      layout.setHorizontalGroup(
//
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
//          .addComponent(jPanel1,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
//      );
//      layout.setVerticalGroup(
//
layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
//          .addComponent(jPanel1,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
//      );
//
//      pack();

```

```

        setLayout(new java.awt.BorderLayout());
        add(jPanell1, java.awt.BorderLayout.CENTER);

    }// </editor-fold>//GEN-END:initComponents

    private void jTextField1ActionPerformed(java.awt.event.ActionEvent evt)
    {//GEN-FIRST:event_jTextField1ActionPerformed
        // TODO add your handling code here:
    }//GEN-LAST:event_jTextField1ActionPerformed

    private void initNavigationActions() {
        jButton2.addActionListener(new java.awt.event.ActionListener() {
            @Override
            public void actionPerformed(java.awt.event.ActionEvent e) {

                frame.showPanel(new Registration(frame, connection));
            }
        });
    }

    //отправка данных
    private void initNetworkActions() {
        jButton1.addActionListener(new java.awt.event.ActionListener() {
            @Override
            public void actionPerformed(java.awt.event.ActionEvent e) {
                String username = jTextField1.getText().trim();
                String password = new
String(jTextField2.getPassword()).trim();

                if (username.isEmpty() || password.isEmpty()) {
                    JOptionPane.showMessageDialog(Login.this, "Заполните все
поля!");
                    return;
                }

                String[] credentials = new String[] { username, password };
                Message loginMsg = new Message(Commands.LOGIN_REQUEST,
credentials);

                try {
                    connection.sendMessage(loginMsg);
                } catch (IOException ex) {

```

```

        throw new RuntimeException(ex);
    }
}

});
}

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JPanel jPanel1;
private javax.swing.JTextField jTextField1;
private javax.swing.JPasswordField jTextField2;
private final MatchFrame frame;
// End of variables declaration//GEN-END:variables
}

```

Файл Mainframe.java

```

package ru.psu.coursework.client.ui;

import ru.psu.coursework.additional.logic.MoveCalculator;
import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.additional.models.Board;
import ru.psu.coursework.additional.models.Cell;
import ru.psu.coursework.additional.models.Dices;
import ru.psu.coursework.client.network.ClientConnection;

import javax.swing.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class Mainframe extends javax.swing.JPanel {

    private javax.swing.JButton jButtonRoll;
    private javax.swing.JButton jButtonBearOff;
    private javax.swing.JLabel jLabelBlack;

```

```

private javax.swing.JLabel jLabelBlackScore;
private javax.swing.JLabel jLabelDice;
private javax.swing.JLabel jLabelOneDice;
private javax.swing.JLabel jLabelTwoDice;
private javax.swing.JLabel jLabelWhite;
private javax.swing.JLabel jLabelWhiteScore;
private javax.swing.JPanel jPanel1;

private final MatchFrame matchFrame;
private final ClientConnection connection;
private final int myColor;
private BoardPanel boardPanel;
private Board currentBoard;
private Dices dices;
private int selectedCell = -1;

public Mainframe(MatchFrame matchFrame, ClientConnection connection, int
myColor, Board initialBoard, Dices initialDices) {
    this.matchFrame = matchFrame;
    this.connection = connection;
    this.myColor = myColor;
    this.currentBoard = initialBoard;
    this.dices = initialDices;

    initComponents();
    initMouseListener();

    initGameActions();

    updateDicesUI(initialDices);

    jButtonRoll.setEnabled(myColor ==
ru.psu.coursework.additional.models.Cell.WHITE);
}

// public Mainframe() {
//     //тестовая доска с начальной расстановкой для проверки отображения
//     currentBoard = new Board();
//
//     initComponents();
//     initMouseListener();
// }

```

```

private void initComponents() {
    jPanel1 = new javax.swing.JPanel();
    boardPanel = new BoardPanel(currentBoard);

    jButtonRoll = new javax.swing.JButton();
    jButtonBearOff = new javax.swing.JButton();
    jLabelWhite = new javax.swing.JLabel();
    jLabelBlack = new javax.swing.JLabel();
    jLabelDice = new javax.swing.JLabel();
    jLabelWhiteScore = new javax.swing.JLabel();
    jLabelBlackScore = new javax.swing.JLabel();
    jLabelOneDice = new javax.swing.JLabel();
    jLabelTwoDice = new javax.swing.JLabel();

    setLayout(new java.awt.BorderLayout());
    jPanel1.setBackground(new java.awt.Color(128, 110, 99));

    jButtonRoll.setBackground(new java.awt.Color(171, 139, 119));
    jButtonRoll.setFont(new java.awt.Font("SimSun-ExtB", 1, 18));
    jButtonRoll.setForeground(new java.awt.Color(86, 57, 39));
    jButtonRoll.setText("Roll");

    jButtonBearOff.setBackground(new java.awt.Color(171, 139, 119));
    jButtonBearOff.setFont(new java.awt.Font("SimSun-ExtB", 1, 14));
    jButtonBearOff.setForeground(new java.awt.Color(86, 57, 39));
    jButtonBearOff.setText("Bear off");
    jButtonBearOff.setEnabled(false);

    jLabelWhite.setFont(new java.awt.Font("SimSun-ExtB", 1, 14));
    jLabelWhite.setText("White");
    jLabelBlack.setFont(new java.awt.Font("SimSun-ExtB", 1, 14));
    jLabelBlack.setText("Black");

    jLabelDice.setFont(new java.awt.Font("SimSun-ExtB", 1, 18));
    jLabelDice.setText("Dice:");

    jLabelWhiteScore.setFont(new java.awt.Font("SimSun-ExtB", 1, 14));
    jLabelBlackScore.setFont(new java.awt.Font("SimSun-ExtB", 1, 14));

    jLabelOneDice.setFont(new java.awt.Font("SimSun-ExtB", 1, 24));
    jLabelTwoDice.setFont(new java.awt.Font("SimSun-ExtB", 1, 24));
}

```



```

        javax.swing.GroupLayout jPanellLayout = new
javax.swing.GroupLayout(jPanell);
        jPanell.setLayout(jPanellLayout);
        jPanellLayout.setHorizontalGroup(

jPanellLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(boardPanel,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
            .addGroup(jPanellLayout.createSequentialGroup()
                .addGap(71, 71, 71)
                .addComponent(jButtonBearOff) // Кнопка Bear
off теперь на своем месте
                .addGap(39, 39, 39)
                .addComponent(jButtonRoll)

            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

            .addGroup(jPanellLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)
                .addComponent(jLabelWhite)

            .addGroup(jPanellLayout.createSequentialGroup()
                .addGap(6, 6, 6)

            .addComponent(jLabelWhiteScore, javax.swing.GroupLayout.PREFERRED_SIZE, 23,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addGap(34, 34, 34)

            .addGroup(jPanellLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)
                .addComponent(jLabelBlack)

            .addGroup(jPanellLayout.createSequentialGroup()
                .addGap(6, 6, 6)

            .addComponent(jLabelBlackScore, javax.swing.GroupLayout.PREFERRED_SIZE, 23,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addGap(78, 78, 78)
                .addComponent(jLabelDice)

```

```

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jLabelOneDice,
javax.swing.GroupLayout.PREFERRED_SIZE,                23,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jLabelTwoDice,
javax.swing.GroupLayout.PREFERRED_SIZE,                23,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(165, 165, 165))

);

jPanell1Layout.setVerticalGroup(

jPanell1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(jPanell1Layout.createSequentialGroup()
                .addComponent(boardPanel,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

.addGroup(jPanell1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

.addGroup(jPanell1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING, false)

                .addComponent(jButtonRoll)
                .addComponent(jButtonBearOff,
javax.swing.GroupLayout.PREFERRED_SIZE,                26,
javax.swing.GroupLayout.PREFERRED_SIZE)

                .addComponent(jLabelDice)
                .addComponent(jLabelOneDice,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

                .addComponent(jLabelTwoDice,
javax.swing.GroupLayout.DEFAULT_SIZE, 33, Short.MAX_VALUE))

.addGroup(jPanell1Layout.createSequentialGroup())
                .addGap(3, 3, 3)

```

```

.addGroup(jPanellLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASELINE)

.addComponent(jLabelWhite)

.addComponent(jLabelBlack))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

.addGroup(jPanellLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING, false)

.addComponent(jLabelWhiteScore,    javax.swing.GroupLayout.DEFAULT_SIZE,    14,
Short.MAX_VALUE)

.addComponent(jLabelBlackScore,      javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)))
        .addGap(9, 9, 9))
    );
    add(jPanell, java.awt.BorderLayout.CENTER);
}

//обработка кликов по доске
private void initMouseListener() {
    boardPanel.addMouseListener(new MouseAdapter() {
        @Override
        public void mousePressed(MouseEvent e) {
            int mouseX = e.getX();
            int mouseY = e.getY();

            int column = mouseX / 70;
            if (column < 0 || column > 11) return;

            int clickedCell;
            if (mouseY > 300) {
                clickedCell = column;
            } else {
                clickedCell = 23 - column;
            }

            System.out.println("Cell selection: " + clickedCell);
        }
    });
}

```

```

        try {
            onSlotSelected(clickedCell);
        } catch (IOException ex) {
            throw new RuntimeException(ex);
        }
    }

});

}

//выбор фишки
private void onSlotSelected(int cellId) throws IOException {
    if (dices == null || !dices.isRolled()) {
        JOptionPane.showMessageDialog(this,
            "You must roll the dice!!!",
            "The move is blocked", JOptionPane.WARNING_MESSAGE);
        return;
    }

    if (selectedCell == -1) {
        Cell cell = currentBoard.getCell(cellId);

        if (cell != null || !cell.isEmpty()) {
            if (cell.getColor() != myColor) return;

            selectedCell = cellId;

            //List<Integer> possibleMoves

            MoveCalculator calculator = new MoveCalculator();
            List<Integer> validMoves = calculator.getPossibleMoves(
                currentBoard, cellId, myColor,
                false, false, dices.isDouble, 0,
dices.getAvailableValues()
            );

            if (validMoves.contains(-1)) {
                jButtonBearOff.setEnabled(true);
                System.out.println("CLIENT: This piece can be discarded
behind the board");
            } else {
                jButtonBearOff.setEnabled(false);
            }
        }
    }
}

```

```

//          List<Integer> testMoves = new ArrayList<>();
//          testMoves.add((cellId + 3) % 24);
//          testMoves.add((cellId + 5) % 24);

        boardPanel.setHighlightedSlots(validMoves);
        boardPanel.repaint();
    }
    } else {
        System.out.println("Move from cell " + selectedCell + " to cell "
+ cellId);

        sendMoveToServer(selectedCell, cellId);
    }
}

private void sendMoveToServer(int from, int to) {
    int[] moveCoords = new int[]{from, to};
    try {
        connection.sendMessage(new Message(Commands.MAKE_MOVE,
moveCoords));
    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
    selectedCell = -1;
    jButtonBearOff.setEnabled(false);
    boardPanel.setHighlightedSlots(new ArrayList<>());
    boardPanel.repaint();
}

//    public static void main(String args[]) {
//        java.awt.EventQueue.invokeLater(() -> new
MainFrame().setVisible(true));
//    }

//roll
private void initGameActions() {
    jButtonRoll.addActionListener(e -> {
        System.out.println("Requesting a dice roll from the server...");

        try {
            connection.sendMessage(new Message(Commands.DICE_ROLL, null));

```

```

        } catch (IOException ex) {
            throw new RuntimeException(ex);
        }
    });

    jButtonBearOff.addActionListener(e -> {
        if (selectedCell != -1) {
            System.out.println("КЛИЕНТ: Игрок выбрал СБРОС фишки из лунки
" + selectedCell);

            sendMoveToServer(selectedCell, -1);
        }
    });
}

    public void refreshGameState(Board newBoard, Dices newDices, int
currentTurnColor) {
        this.currentBoard = newBoard;
        this.dices = newDices;

        if (boardPanel != null) {
            boardPanel.setBoard(newBoard);
            boardPanel.repaint();
        }

        //        boardPanel.repaint();

        //
        int whitePiecesOnBoard = 0;
        int blackPiecesOnBoard = 0;

        for (int i = 0; i < 24; i++) {
            Cell cell = newBoard.getCell(i);
            if (cell != null && !cell.isEmpty()) {
                if
                    (cell.getColor()
                        ==
ru.psu.coursework.additional.models.Cell.WHITE) {
                    whitePiecesOnBoard += cell.getCount();
                }
                else
                    if
                        (cell.getColor()
                            ==
ru.psu.coursework.additional.models.Cell.BLACK) {
                    blackPiecesOnBoard += cell.getCount();
                }
            }
        }
    }
}

```

```

        }
    }

    int whiteScore = 15 - whitePiecesOnBoard;
    int blackScore = 15 - blackPiecesOnBoard;

    if (jLabelWhiteScore != null)
        jLabelWhiteScore.setText(String.valueOf(whiteScore));

    if (jLabelBlackScore != null)
        jLabelBlackScore.setText(String.valueOf(blackScore));

    //

    if (newDices != null) {
        System.out.println("CLIENT INTERFACE: Cubes arrived from the
network: "
            + newDices.getDiceOne() + " and " + newDices.getDiceTwo());
    }

    updateDicesUI(newDices);

    boolean isMyTurn = (currentTurnColor == myColor);
    //        jButtonRoll.setEnabled(isMyTurn && (newDices == null ||
!newDices.isRolled()));
    if (!isMyTurn || (newDices != null && newDices.getDiceOne() > 0)) {
        jButtonRoll.setEnabled(false);
        System.out.println("CLIENT INTERFACE: Roll button is disabled");
    } else {
        jButtonRoll.setEnabled(true);
        System.out.println("CLIENT INTERFACE: Roll button available for
throwing");
    }
}

private void updateDicesUI(Dices dices) {
    if (dices != null && dices.getDiceOne() > 0 && dices.getDiceTwo() > 0)
    {
        if (jLabelOneDice == null || jLabelTwoDice == null) {
            System.err.println("UI ERROR: Cube text labels are null!");
        }
    }
}

```

```

        return;
    }

    jLabelOneDice.setText(String.valueOf(dices.getDiceOne()));
    jLabelTwoDice.setText(String.valueOf(dices.getDiceTwo()));
    System.out.println("The interface displayed dices: " +
dices.getDiceOne() + " : " + dices.getDiceTwo());
    //System.out.println("КЛИЕНТ ИНТЕРФЕЙС: Текст лейблов успешно
изменен на цифры!");
    } else {
        jLabelOneDice.setText("-");
        jLabelTwoDice.setText("-");
        //System.out.println("КЛИЕНТ ИНТЕРФЕЙС: Кубики пустые, отрисованы
прочерки.");
    }
}
}

```

Файл MatchFrame.java

```

package ru.psu.coursework.client.ui;

import ru.psu.coursework.client.network.ClientConnection;

import javax.swing.*;

public class MatchFrame extends JFrame {

    private ClientConnection connection;

    public MatchFrame() {
        setTitle("Backgammon");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(840, 660);
        setResizable(false);
        setLocationRelativeTo(null);

        try {
            connection = new ClientConnection("localhost", 8765, this);
            connection.connect();

            Thread networkThread = new Thread(connection);

```



```

        networkThread.start();

    } catch (Exception e) {
        JOptionPane.showMessageDialog(this,
            "Failed to connect to the server!",
            "Network error!!!", JOptionPane.ERROR_MESSAGE);

        System.exit(0);
    }

    showPanel(new Login(this, connection));
}

public void showPanel(JPanel panel) {
    getContentPane().removeAll();
    add(panel);
    revalidate();
    repaint();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new MatchFrame().setVisible(true));
}
}

```

Файл MenuPanel.java

```

package ru.psu.coursework.client.ui;

import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.client.network.ClientConnection;

import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class MenuPanel extends javax.swing.JPanel {

    private final MatchFrame frame;

    private javax.swing.JButton jButtonCreateGame;
    private javax.swing.JButton jButtonJoin;

```

```

private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabelPhoto;
private javax.swing.JMenu jMenuAbout;
private javax.swing.JMenuBar jMenuBar;
private javax.swing.JMenuItem jMenuItemLogout;
private javax.swing.JMenu jMenuSettings;
private javax.swing.JPanel jPanel1;
private ClientConnection connection;

public MenuPanel(MatchFrame frame, ClientConnection connection) {
    this.frame = frame;
    this.connection = connection;
    initComponents();
    initNavigationActions();
}

private void initComponents() {
    jPanel1 = new javax.swing.JPanel();
    jButtonCreateGame = new javax.swing.JButton();
    jButtonJoin = new javax.swing.JButton();
    jLabel1 = new javax.swing.JLabel();
    jLabelPhoto = new javax.swing.JLabel();
    jMenuBar = new javax.swing.JMenuBar();
    jMenuSettings = new javax.swing.JMenu();
    jMenuItemLogout = new javax.swing.JMenuItem();
    jMenuAbout = new javax.swing.JMenu();

    // Главный менеджер для всего экрана лобби
    setLayout(new java.awt.BorderLayout());

    jPanel1.setBackground(new java.awt.Color(128, 110, 99));

    jButtonCreateGame.setBackground(new java.awt.Color(171, 139, 119));
    jButtonCreateGame.setFont(new java.awt.Font("SimSun-ExtB", 1, 18));
    jButtonCreateGame.setForeground(new java.awt.Color(86, 57, 39));
    jButtonCreateGame.setText("Create the game");

    jButtonJoin.setBackground(new java.awt.Color(171, 139, 119));
    jButtonJoin.setFont(new java.awt.Font("SimSun-ExtB", 1, 18));
    jButtonJoin.setForeground(new java.awt.Color(86, 57, 39));
    jButtonJoin.setText("Join the game");

```

```

jLabel1.setFont(new java.awt.Font("SimSun-ExtB", 1, 36));
jLabel1.setText("Backgammon");

jLabelPhoto.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/ru/psu/coursework/other/CQs5VP
CxauzDaoRD5wY01dc7Dvzlf15szKr1tOGESLMIDfcM4z8n4r59_wmTpmzuur6WGcCP2MJXn234-
Eh-XLH0.jpg")));

javax.swing.GroupLayout jPanel1Layout = new
javax.swing.GroupLayout(jPanel1);
jPanel1.setLayout(jPanel1Layout);
jPanel1Layout.setHorizontalGroup(

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(jPanel1Layout.createSequentialGroup()
        .addGap(41, 41, 41)

.addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.TRAILING)

                                .addComponent(jLabel1)
                                .addComponent(jLabelPhoto))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 66,
Short.MAX_VALUE)

.addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

                                .addComponent(jButtonCreateGame,
javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.PREFERRED_SIZE, 200,
javax.swing.GroupLayout.PREFERRED_SIZE)

                                .addComponent(jButtonJoin,
javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.PREFERRED_SIZE, 200,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(48, 48, 48))
    );
jPanel1Layout.setVerticalGroup(

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(jPanel1Layout.createSequentialGroup()

```

```

.addGroup(jPanell1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

.addGroup(jPanell1Layout.createSequentialGroup())
                                .addGap(244, 244, 244)

.addComponent(jButtonCreateGame, javax.swing.GroupLayout.PREFERRED_SIZE, 40,
javax.swing.GroupLayout.PREFERRED_SIZE)
                                .addGap(65, 65, 65)
                                .addComponent(jButtonJoin,
javax.swing.GroupLayout.PREFERRED_SIZE,                                40,
javax.swing.GroupLayout.PREFERRED_SIZE))

.addGroup(jPanell1Layout.createSequentialGroup())
                                .addGap(48, 48, 48)
                                .addComponent(jLabel1)
                                .addGap(61, 61, 61)
                                .addComponent(jLabelPhoto)))
                                .addContainerGap(106, Short.MAX_VALUE))

);

jMenuSettings.setText("Settings");
jMenuItemLogout.setText("Logout");
jMenuSettings.add(jMenuItemLogout);
jMenuBar.add(jMenuSettings);

jMenuAbout.setText("About");
jMenuBar.add(jMenuAbout);

add(jMenuBar, java.awt.BorderLayout.NORTH);
add(jPanell1, java.awt.BorderLayout.CENTER);
}

private void initNavigationActions() {
    jMenuItemLogout.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            System.out.println("Logging out... Returning to login
window.");

            frame.showPanel(new Login(frame, connection));

```

```

        }
    });

    jButtonCreateGame.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            frame.setSize(400, 300);
            frame.setLocationRelativeTo(null);
            frame.showPanel(new Create(frame, connection));
        }
    });

    jButtonJoin.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            frame.setSize(400, 300);
            frame.setLocationRelativeTo(null);
            frame.showPanel(new Join(frame, connection));
        }
    });
}
}

```

Файл Registration.java

```

package ru.psu.coursework.client.ui;

import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.client.network.ClientConnection;

import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.IOException;

public class Registration extends javax.swing.JPanel {

    private final ClientConnection connection;
    private final MatchFrame frame;
    private javax.swing.JButton jButtonCancel;
    private javax.swing.JButton jButtonSignUp;

```

```

private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JPanel jPanel1;
private javax.swing.JTextField jTextFieldCreateLogin;
private javax.swing.JPasswordField jPasswordFieldCreatePassword; //
ЗАМЕНЕНО НА БЕЗОПАСНЫЙ КЛАСС

//private ClientConnector connector;

// public Registration() {
//     initComponents();
//     initNetworkActions();
// }s

public Registration(MatchFrame frame, ClientConnection connection) {
    this.frame = frame;
    this.connection = connection;
    initComponents();
    initNetworkActions();
}

private void initComponents() {

    jPanel1 = new javax.swing.JPanel();
    jTextFieldCreateLogin = new javax.swing.JTextField();
    jPasswordFieldCreatePassword = new javax.swing.JPasswordField(); //
Заменено поле
    jLabel1 = new javax.swing.JLabel();
    jLabel2 = new javax.swing.JLabel();
    jButtonSignUp = new javax.swing.JButton();
    jButtonCancel = new javax.swing.JButton();

    setLayout(new java.awt.BorderLayout());

    jPanel1.setBackground(new java.awt.Color(128, 110, 99));

    jLabel1.setFont(new java.awt.Font("SimSun-ExtB", 1, 14));
    jLabel1.setText("Create a login");

    jLabel2.setFont(new java.awt.Font("SimSun-ExtB", 1, 14));

```

```

jLabel2.setText("Create a password");

jButtonSignUp.setBackground(new java.awt.Color(171, 139, 119));
jButtonSignUp.setFont(new java.awt.Font("SimSun-ExtB", 1, 18));
jButtonSignUp.setForeground(new java.awt.Color(86, 57, 39));
jButtonSignUp.setText("Sign Up");

jButtonCancel.setBackground(new java.awt.Color(171, 139, 119));
jButtonCancel.setFont(new java.awt.Font("SimSun-ExtB", 0, 12)); //
NOI18N
jButtonCancel.setForeground(new java.awt.Color(86, 57, 39));
jButtonCancel.setText("Cancel");

javax.swing.GroupLayout jPanel1Layout = new
javax.swing.GroupLayout(jPanel1);
jPanel1.setLayout(jPanel1Layout);
jPanel1Layout.setHorizontalGroup(

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(jPanel1Layout.createSequentialGroup()
        .addContainerGap()
        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jTextFieldCreateLogin)
            .addComponent(jPasswordFieldCreatePassword,
                javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addComponent(jLabel1,
                javax.swing.GroupLayout.PREFERRED_SIZE, 126,
                javax.swing.GroupLayout.PREFERRED_SIZE)
        )
    )
);

```

```

.addComponent(jLabel2,          javax.swing.GroupLayout.PREFERRED_SIZE,          145,
javax.swing.GroupLayout.PREFERRED_SIZE)))

.addGroup(jPanell1Layout.createSequentialGroup())
                                .addGap(319, 319, 319)
                                .addComponent(jButtonSignUp,
javax.swing.GroupLayout.PREFERRED_SIZE,          190,
javax.swing.GroupLayout.PREFERRED_SIZE))

.addGroup(jPanell1Layout.createSequentialGroup())
                                .addGap(371, 371, 371)

.addComponent(jButtonCancel))) // Теперь скобки закрываются математически верно
                                .addContainerGap(253, Short.MAX_VALUE))

);
jPanell1Layout.setVerticalGroup(

jPanell1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                                .addGroup(jPanell1Layout.createSequentialGroup())
                                    .addGap(151, 151, 151)
                                    .addComponent(jLabel1)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                                .addComponent(jTextFieldCreateLogin,
javax.swing.GroupLayout.PREFERRED_SIZE,  javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
                                    .addGap(47, 47, 47)
                                    .addComponent(jLabel2)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                                .addComponent(jPasswordFieldCreatePassword,
javax.swing.GroupLayout.PREFERRED_SIZE,  javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
                                    .addGap(82, 82, 82)
                                    .addComponent(jButtonSignUp,
javax.swing.GroupLayout.PREFERRED_SIZE,          31,
javax.swing.GroupLayout.PREFERRED_SIZE)
                                    .addGap(43, 43, 43)
                                    .addComponent(jButtonCancel)
                                    .addContainerGap(190, Short.MAX_VALUE))

);

```



```

        add(jPanell1, java.awt.BorderLayout.CENTER);
    }

    private void initNetworkActions() {
        jButtonSignUp.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                String username = jTextFieldCreateLogin.getText().trim();
                String password = new
String(jTextFieldCreatePassword.getPassword()).trim();

                if (username.isEmpty() || password.isEmpty()) {
                    JOptionPane.showMessageDialog(Registration.this,
                        "Please fill in all fields to register!",
"ERROR!!!", JOptionPane.WARNING_MESSAGE);
                    return;
                }

                //логин и пароль
                String[] credentials = new String[] { username, password };

                Message regMsg = new Message(Commands.REGISTRATION_REQUEST,
credentials);

                System.out.println("Sending a registration request to the
server for: " + username);

                try {
                    connection.sendMessage(regMsg);
                } catch (IOException ex) {
                    throw new RuntimeException(ex);
                }
            }
        });

        jButtonCancel.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {

```

```

        System.out.println("Registration canceled. Returning to login
screen.");

        frame.showPanel(new Login(frame, connection));
    }
    });
}
}

```

Листинг сервера

Файл Main.java

```

package ru.psu.coursework.server.main;

import ru.psu.coursework.server.database.DatabaseManager;
import ru.psu.coursework.server.network.LobbyManager;
import ru.psu.coursework.server.network.PlayerConnection;

import java.net.*;
import java.io.*;

public class Main {
    private static final int PORT = 8765;

    public static void main(String[] args) throws IOException {
        System.out.println("Starting server...");

        DatabaseManager database = new DatabaseManager();
        LobbyManager lobby = new LobbyManager();

        ServerSocket serverSocket = new ServerSocket(PORT);

        try {
            System.out.println("Server successfully started and listening on
port " + PORT);

            while (true) {
                Socket socket = serverSocket.accept();

```

```

        PlayerConnection connection = new PlayerConnection(socket,
database, lobby);

        Thread thread = new Thread(connection);
        thread.start();

    }
    } finally {
        serverSocket.close();
    }
}
}

```

Файл DatabaseManager.java

```

package ru.psu.coursework.server.database;

import java.sql.*;

public class DatabaseManager {

    private static final String URL =
"jdbc:postgresql://localhost:5432/backgammon";
    private static final String USER = "postgres";
    private static final String PASSWORD = "admin";

    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    public boolean registerUser(String username, String password) {
        String sql = "INSERT INTO users (username, password) VALUES (?, ?)";

        try (Connection conn = getConnection()) {
            conn.setAutoCommit(false);

            try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
                pstmt.setString(1, username);
                pstmt.setString(2, password);

                int rowsInserted = pstmt.executeUpdate();
            }
        }
    }
}

```

```

        if (rowsInserted > 0) {
            conn.commit();
            return true;
        }
    } catch (SQLException e) {
        conn.rollback();
        if ("23505".equals(e.getSQLState())) {
            System.out.println("Логин занят.");
        } else {
            e.printStackTrace();
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}

return false;
}

public boolean loginUser(String username, String password) {
    String sql = "SELECT password FROM users WHERE username = ?";

    try (Connection connection = getConnection();
        PreparedStatement pstmt = connection.prepareStatement(sql)) {

        pstmt.setString(1, username);

        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) { //если пользователь существует
                String storedHash = rs.getString("password");//правильный
пароль

                return storedHash.equals(password);
            }
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return false;
}

```

```

    }

}

```

Файл LobbyManager.java

```

package ru.psu.coursework.server.network;

import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;

import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

public class LobbyManager {

    private final List<PlayerConnection> players = new
CopyOnWriteArrayList<>();
    private final Map<String, Room> activeRooms = new ConcurrentHashMap<String,
Room>();

    public synchronized void addPlayer(PlayerConnection player) {
        players.add(player);
    }

    public synchronized void createRoom(PlayerConnection creator, String
roomCode) {
        if (roomCode == null) {
            creator.send(new Message(Commands.ERROR, "Room code cannot be
empty!"));
            return;
        }

        String code = roomCode.trim().toUpperCase();

        if (activeRooms.containsKey(code)) {
            creator.send(new Message(Commands.ERROR, "The room code is already
taken! Please create another one!!!"));

            return;
        }
    }
}

```

```

Room newRoom = new Room(code, creator);

activeRooms.put(code, newRoom);
players.remove(creator);
creator.setCurrentRoom(newRoom);

    creator.send(new Message(Commands.CREATE_ROOM, "The room " + code + "
has been successfully created. Waiting for the opponent..."));
    System.out.println("Created the room " + code);
}

public synchronized void joinRoom(PlayerConnection guest, String roomCode)
{
    if (roomCode == null) {
        guest.send(new Message(Commands.ERROR, "Room code cannot be
empty!"));

        return;
    }

    String code = roomCode.trim().toUpperCase();
    Room room = activeRooms.get(code);

    if (room == null) {
        guest.send(new Message(Commands.ERROR, "The room " + code + " does
not exist!"));

        return;
    }

    if (room.isFull()) {
        guest.send(new Message(Commands.ERROR, "The room " + code + " is
already full!"));

        return;
    }

    room.addOpponent(guest);
    players.remove(guest);

```

```

        guest.setCurrentRoom(room);

        System.out.println("Player " + guest.getUsername() + " joined the room
successfully!");

        room.start();
    }

    public synchronized void removePlayer(PlayerConnection player) {
        players.remove(player);
    }
}

```

Файл PlayerConnection.java

```

package ru.psu.coursework.server.network;

import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.additional.models.Cell;
import ru.psu.coursework.additional.models.Dices;
import ru.psu.coursework.server.database.DatabaseManager;

import java.io.*;
import java.net.*;

public class PlayerConnection implements Runnable {

    private final Socket socket;
    private ObjectInputStream in;
    private ObjectOutputStream out;
    private DatabaseManager database;
    private LobbyManager lobby; //добавить
    private String username = null;
    private boolean isConnected = true;
    private Room currentRoom = null;

    public PlayerConnection(Socket socket, DatabaseManager database,
LobbyManager lobby) {
        this.socket = socket;
        this.database = database;
        this.lobby = lobby;
    }
}

```

```

    }

    @Override
    public void run() {
        try {
            this.out = new ObjectOutputStream(socket.getOutputStream());
            this.in = new ObjectInputStream(socket.getInputStream());

            System.out.println("Connection Established");

            while (isConnected) {
                Message message = (Message) in.readObject();

                processMessage(message);
            }

        } catch (EOFException | SocketException e) {
            System.out.println("Player " + username + " disconnected");
        } catch (IOException | ClassNotFoundException e) {
            throw new RuntimeException(e);
        } catch (Exception e) {
            System.err.println("Critical error while processing message: " +
e.getMessage());
            e.printStackTrace();
        } finally {
            closeConnection();
        }
    }

    private void processMessage(Message message) {
        if (message.getCommand() == Commands.REGISTRATION_REQUEST) {
            String[] credentials = (String[]) message.getData();
            String regLogin = credentials[0];
            String regPassword = credentials[1];

            boolean isRegOk = database.registerUser(regLogin, regPassword);

            if (isRegOk) {
                send(new Message(Commands.REGISTRATION_REQUEST, "Registration
successful!"));
            }
        }
    }

```



```

        System.out.println("User    registered    successfully:    "    +
regLogin);

        } else {
            send(new Message(Commands.ERROR, "Login is already taken!"));
        }

    } else if (message.getCommand() == Commands.LOGIN_REQUEST) {
//        System.out.println("LOGIN_REQUEST received");
//        System.out.println("Message data: " + message.getData());

        String[] credentials = (String[]) message.getData();

//        if (credentials == null || credentials.length < 2) {
//            System.out.println("ERROR: Invalid credentials array");
//            send(new Message(Commands.ERROR, "Invalid login data"));
//            return;
//        }
//        //
        String loginName = credentials[0];
        String password = credentials[1];

//        System.out.println("Login: '" + loginName + "', Password: '" +
password + "'");

        boolean isAuthOk = database.loginUser(loginName, password);

        if (isAuthOk) {
            System.out.println("loginName from client: '" + loginName +
"");

            this.username = loginName;

            lobby.addPlayer(this);

            Message successMsg = new Message();
            successMsg.setCommand(Commands.AUTHENTICATION_SUCCESS);
            successMsg.setData(loginName);
            send(successMsg);

//            send(new Message(Commands.AUTHENTICATION_SUCCESS, loginName));

```

```

        System.out.println("User authenticated: " + loginName);

    } else {
        send(new Message(Commands.ERROR, "Invalid login or
password!"));
    }

    } else if (message.getCommand() == Commands.AUTHENTICATION_SUCCESS) {
        System.out.println("Warning: Received outbound-only command from
client!!!");

    } else if (message.getCommand() == Commands.CREATE_ROOM) {
        String roomCode = (String) message.getData();

        lobby.createRoom(this, roomCode);

    } else if (message.getCommand() == Commands.JOIN_ROOM) {
        String roomCode = (String) message.getData();

        lobby.joinRoom(this, roomCode);

    } else if (message.getCommand() == Commands.GAME_START) {
        System.out.println("Warning: Received outbound-only command from
client!!!");

    } else if (message.getCommand() == Commands.DICE_ROLL) {
        Room room = this.getCurrentRoom();
        if (room != null) {

            System.out.println("The server accepted the Roll request from
the player: " + this.getUsername());
            System.out.println("Now the color is moving: " +
room.getCurrentTurnColor());
            System.out.println("White player on the server: " +
(room.getWhitePlayer() != null ? room.getWhitePlayer().getUsername() :
>null));
            System.out.println("Black player on the server: " +
(room.getBlackPlayer() != null ? room.getBlackPlayer().getUsername() :
>null));
        }
    }
}

```

```

        if (room.getCurrentTurnColor() == Cell.WHITE && this !=
room.getWhitePlayer()) {
            send(new Message(Commands.ERROR, "Now it's White's turn!
You can't roll the dice!!!"));
            return;
        }
        if (room.getCurrentTurnColor() == Cell.BLACK && this !=
room.getBlackPlayer()) {
            send(new Message(Commands.ERROR, "Now it's Black's turn!
You can't roll the dice!!!"));
            return;
        }

        Dices roomDices = room.getDices();
        roomDices.roll();

        System.out.println("SERVER GENERATED: " +
roomDices.getDiceOne() + " and " + roomDices.getDiceTwo());

        //room.getDices().roll();

        Object[] gameData = new Object[] { room.getBoard(),
room.getDices(), room.getCurrentTurnColor() };
        Message updateMsg = new Message(Commands.UPDATE_BOARD,
gameData);

        if (room.getWhitePlayer() != null) {
            room.getWhitePlayer().resetSocketCache();
            room.getWhitePlayer().send(updateMsg);
        }
        if (room.getBlackPlayer() != null) {
            room.getWhitePlayer().resetSocketCache();
            room.getBlackPlayer().send(updateMsg);
        }

        //            room.getWhitePlayer().send(updateMsg);
        //            room.getBlackPlayer().send(updateMsg);
    }

} else if (message.getCommand() == Commands.MAKE_MOVE) {
    if (currentRoom != null) {
        int[] moveCoords = (int[]) message.getData();

```

```

        int from = moveCoords[0];
        int to = moveCoords[1];

        currentRoom.operatePlayersMove(this, from, to);

    } else {
        send(new Message(Commands.ERROR, "You are not in the active
game room!"));
    }

    } else if (message.getCommand() == Commands.UPDATE_BOARD) {
        System.out.println("Warning: Received outbound-only command from
client!!!");
    } else {
        send(new Message(Commands.ERROR, "Unknown server command code!"));
    }
}

public void setCurrentRoom(Room room) {
    this.currentRoom = room;
}

public Room getCurrentRoom() {
    return currentRoom;
}

public String getUsername() {
    return username;
}

public synchronized void send(Message message) {
    try {
        if (out != null && !socket.isClosed()) {
            out.reset();
            out.writeObject(message);
            out.flush();
        }
    } catch (IOException e) {
        System.err.println("Error sending message to player " +
e.getMessage());
    }
}

```

```

    }
}

private void closeConnection() {
    isConnected = false;

    if (lobby != null)
        lobby.removePlayer(this);

    try {
        if (in != null) in.close();
        if (out != null) out.close();
        if (!socket.isClosed()) socket.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

public synchronized void resetSocketCache() {
    try {
        if (out != null && !socket.isClosed()) {
            out.reset();
        }
    } catch (IOException e) {
        System.err.println("Socket cache flush error: " + e.getMessage());
    }
}
}

```

Файл Room.java

```

package ru.psu.coursework.server.network;

import ru.psu.coursework.additional.logic.MoveCalculator;
import ru.psu.coursework.additional.logic.MoveValidator;
import ru.psu.coursework.additional.logic.WinConditionChecker;
import ru.psu.coursework.additional.logic.TypeOfWin;
import ru.psu.coursework.additional.messaging.Commands;
import ru.psu.coursework.additional.messaging.Message;
import ru.psu.coursework.additional.models.Board;
import ru.psu.coursework.additional.models.Cell;
import ru.psu.coursework.additional.models.Dices;

```

```

public class Room {

    private final String roomCode;
    private final PlayerConnection white;
    private PlayerConnection black;
    private final Board board;
    private final Dices dices;
    private int currentTurnColor;
    private boolean isGameStarted = false;
    private boolean whiteHeadMoveDone = false;
    private boolean blackHeadMoveDone = false;
    private boolean isWhiteFirstMove = true;
    private boolean isBlackFirstMove = true;
    private int headPiecesTaken = 0;

    public Room(String roomCode, PlayerConnection creator) {
        this.roomCode = roomCode;
        this.white = creator;
        this.board = new Board();
        this.dices = new Dices();
    }

    public void addOpponent(PlayerConnection opponent) {
        this.black = opponent;
    }

    public boolean isFull() {
        return black != null;
    }

    public void start() {
        this.isGameStarted = true;
        this.currentTurnColor = Cell.WHITE;

        //this.dices.roll();

        Object[] whiteData = new Object[] { Cell.WHITE, dices, board };
        white.send(new Message(Commands.GAME_START, whiteData));

        Object[] blackData = new Object[] { Cell.BLACK, dices, board };
        black.send(new Message(Commands.GAME_START, blackData));
    }
}

```

```

        System.out.println("Match started!!!");
    }

    public synchronized void operatePlayersMove(PlayerConnection player, int
from, int to) {
        if (!isGameStarted) {
            player.send(new Message(Commands.ERROR, "The match hasn't started
yet!"));

            return;
        }

        if (currentTurnColor == Cell.WHITE && player != white) {
            player.send(new Message(Commands.ERROR, "It's not your move
now!"));

            return;
        }

        if (currentTurnColor == Cell.BLACK && player != black) {
            player.send(new Message(Commands.ERROR, "It's not your move
now!"));

            return;
        }

        //int diceValue = (to - from + 24) % 24;
        int diceValue = -1;
        if (to == -1) {
            int exactNeed;
            if (currentTurnColor == Cell.WHITE)
                exactNeed = 24 - from;
            else
                exactNeed = 12 - from;

            if (dices.getAvailableValues().contains(exactNeed)) {
                diceValue = exactNeed;
            } else {
                for (int availableDice : dices.getAvailableValues()) {
                    if (availableDice > exactNeed) {
                        boolean hasOlderPieces = false;

```

```

        if (currentTurnColor == Cell.WHITE) {
            for (int i = 18; i < from; i++) {
                Cell checkCell = board.getCell(i);
                if (checkCell != null && !checkCell.isEmpty()
&& checkCell.getColor() == Cell.WHITE) {
                    hasOlderPieces = true;
                    break;
                }
            }
        } else {
            for (int i = 6; i < from; i++) {
                Cell checkCell = board.getCell(i);
                if (checkCell != null && !checkCell.isEmpty()
&& checkCell.getColor() == Cell.BLACK) {
                    hasOlderPieces = true;
                    break;
                }
            }
        }

        if (!hasOlderPieces) {
            diceValue = availableDice;
            break;
        }
    }
}

if (diceValue == -1) {
    player.send(new Message(Commands.ERROR, "You don't have a
matching dice for this bear off!"));
    return;
}
} else {
    diceValue = (to - from + 24) % 24;

    if (!dices.getAvailableValues().contains(diceValue)) {
        player.send(new Message(Commands.ERROR, "You don't have a dice
with value " + diceValue + "!"));
        return;
    }
}

```



```

    }

    //      if (!dices.getAvailableValues().contains(diceValue)) {
    //          player.send(new Message(Commands.ERROR, "You don't have a dice
    with value " + diceValue + "!"));
    //      return;
    //      }

    //доступное значение

    MoveValidator validator = new MoveValidator();

    boolean currentHeadMoveDone = (currentTurnColor == Cell.WHITE) ?
    whiteHeadMoveDone : blackHeadMoveDone;
    boolean currentFirstMove = (currentTurnColor == Cell.WHITE) ?
    isWhiteFirstMove : isBlackFirstMove;
    boolean isDouble = dices.getDiceOne() == dices.getDiceTwo();

    boolean canMove = validator.canMove(
        board, from, to, currentTurnColor, currentHeadMoveDone,
        diceValue, currentFirstMove, isDouble, headPiecesTaken);

    if (!canMove) {
        player.send(new Message(Commands.ERROR, "This move is
    impossible!"));

        return;
    }

    //validator.move(board, from, to);
    if (to == -1) {
        Cell targetCell = board.getCell(from);
        //      targetCell.setCount(targetCell.getCount() - 1);
        //      if (targetCell.getCount() == 0)
        //          targetCell.removePiece();
        targetCell.removePiece(); // ИСПРАВЛЕНО: метод сам уменьшит count
    и сотрет цвет при 0!
        System.out.println("SERVER: The piece has been successfully removed
    from cell " + from);
    }

```

```

        System.out.println("SERVER: The " + currentTurnColor + " piece has
been successfully removed from the board!");
    } else {
        validator.move(board, from, to);
    }

    dices.takeDice(diceValue);

    boolean isFromHead = (currentTurnColor == Cell.WHITE && from == 0) ||
(currentTurnColor == Cell.BLACK && from == 12);
    if (isFromHead) {
        if (currentTurnColor == Cell.WHITE)
            whiteHeadMoveDone = true;
        else
            blackHeadMoveDone = true;

        headPiecesTaken++;
    }

    WinConditionChecker win = new WinConditionChecker();

    if (win.isGameOver(board)) {
        endMatch(win);

        return;
    }

    //
    if (dices.getAvailableValues().isEmpty()) {
        System.out.println("SERVER: Player is out of dice. Pass the
turn!");

        changeTurn();
        return;
    }

    MoveCalculator calculator = new MoveCalculator();

    boolean isRolled = dices.isRolled() && calculator.hasAnyValidMoves(
        board, currentTurnColor, currentHeadMoveDone,
currentFirstMove, isDouble, headPiecesTaken, dices.getAvailableValues()
    );

```

```

        if (!isRolled) {
            System.out.println("SERVER: Nowhere to go!");
            changeTurn();
        }
        else {
            System.out.println("SERVER: There are available moves!");
            broadcastState();
        }
    }

    private void endMatch(WinConditionChecker win) {
        isGameStarted = false;

        int winner = win.getWinnerColor(board);
        TypeOfWin typeOfWin = win.checkWin(board, winner);

        String winnerName = (winner == Cell.WHITE) ? white.getUsername() :
black.getUsername();
        String endMessage = "Game over! The player " + winnerName + " has
won!!!";

        white.send(new Message(Commands.ERROR, endMessage));
        black.send(new Message(Commands.ERROR, endMessage));

    }

    private void changeTurn() {
        if (currentTurnColor == Cell.WHITE) {
            isWhiteFirstMove = false;
            currentTurnColor = Cell.BLACK;
        } else {
            isBlackFirstMove = false;
            currentTurnColor = Cell.WHITE;
        }

        whiteHeadMoveDone = false;
        blackHeadMoveDone = false;
        headPiecesTaken = 0;
    }

```

```

        //this.dices = new Dices();

        dices.clear();

        //dices.roll();

        broadcastState();
    }

    private void broadcastState() {
        Object[] state = new Object[] { board, dices, currentTurnColor };
        Message message = new Message(Commands.UPDATE_BOARD, state);

        //        white.send(message);
        //        black.send(message);

        if (white != null) {
            white.resetSocketCache();
            white.send(message);
        }
        if (black != null) {
            black.resetSocketCache();
            black.send(message);
        }

        System.out.println("The game state has been sent successfully");
    }

    public PlayerConnection getWhitePlayer() { return white; }
    public PlayerConnection getBlackPlayer() { return black; }
    public Board getBoard() { return board; }
    public Dices getDices() { return dices; }
    public int getCurrentTurnColor() { return currentTurnColor; }

}

```

Листинг логической части

Файл Message.java

```
package ru.psu.coursework.additional.messaging;

import java.io.Serializable;

public class Message implements Serializable {

    private int command;
    private Object data;
    private String errorMessage;

    public Message() {

    }

    public Message(int command, Object data) {
        this.command = command;
        this.data = data;
        this.errorMessage = null;
    }

    public Message(int command, String errorMessage) { //для отправки ошибки
        this.command = command;
        this.data = null;
        this.errorMessage = errorMessage;
    }

    public int getCommand() {
        return command;
    }

    public void setCommand(int command) {
        this.command = command;
    }

    public Object getData() {
        return data;
    }

    public void setData(Object data) {
        this.data = data;
    }
}
```

```

    }

    public String getErrorMessage() {
        return errorMessage;
    }

    public void setErrorMessage(String errorMessage) {
        this.errorMessage = errorMessage;
    }
}

```

Файл Mars.java

```

package ru.psu.coursework.additional.logic;

public class Mars implements TypeOfWin {
    public String getName() {
        return "Mars";
    }

    public int getScorePoints() {
        return 2;
    }

    public String getDescription(String player) {
        return "Player " + player + " smashed the opponent!!!!!!";
    }
}

```

Файл MoveCalculator.java

```

package ru.psu.coursework.additional.logic;

import ru.psu.coursework.additional.models.*;

import java.util.ArrayList;
import java.util.List;

public class MoveCalculator {

    public MoveCalculator() {

    }
}

```

```

    public List<Integer> getPossibleMoves(Board board, int from, int
playersColor, boolean headMoveDone,
                                boolean isFirstGameMove, boolean isDouble,
int headPiecesTaken, List<Integer> availableDices) {

    List<Integer> possibleMoves = new ArrayList<Integer>();
    MoveValidator validator = new MoveValidator();

    for (int dice : availableDices){
        int to = (from + dice) % 24;

        if (validator.canMove(board, from, to, playersColor, headMoveDone,
dice,
                                isFirstGameMove, isDouble, headPiecesTaken)){
            if (!possibleMoves.contains(to))
                possibleMoves.add(to);
        }

    }

    //выброс с доски
    for (int dice : availableDices){
        if (validator.canRemoveFromBoard(board, playersColor, from, dice))
        {
            if (!possibleMoves.contains(-1))
                possibleMoves.add(-1); //флаг на выход
        }
    }

    return possibleMoves;

}

    public List<Integer> getMovablePieces(Board board, int playersColor,
boolean headMoveDone, boolean isFirstGameMove,
                                boolean isDouble, int
headPiecesTaken, List<Integer> availableDices) {

    List<Integer> movablePositions = new ArrayList<Integer>();
    MoveValidator validator = new MoveValidator();

```

```

        for (int i = 0; i < 24; i++){
            Cell cell = board.getCell(i);

            if (!cell.isEmpty() && cell.getColor() == playersColor) {
                List<Integer> targets = getPossibleMoves(board, i,
playersColor, headMoveDone, isFirstGameMove, isDouble, headPiecesTaken,
availableDices);

                if (!targets.isEmpty())
                    movablePositions.add(i);
            }
        }

        return movablePositions;
    }

    public boolean hasAnyValidMoves(Board board, int playersColor, boolean
headMoveDone, boolean isFirstGameMove,
                                boolean isDouble, int headPiecesTaken,
List<Integer> availableDices) {

        List<Integer> movablePieces = getMovablePieces(board, playersColor,
headMoveDone, isFirstGameMove, isDouble, headPiecesTaken, availableDices);

        if (movablePieces.isEmpty()) return false;

        return true;
    }
}

```

Файл MoveValidator.java

```

package ru.psu.coursework.additional.logic;

import ru.psu.coursework.additional.models.*;

public class MoveValidator {

    public MoveValidator() {

    }
}

```



```

    public boolean canMove(Board board, int from, int to, int playersColor,
boolean headMoveDone, int diceValue,
                                boolean isFirstGameMove, boolean isDouble, int
headPiecesTaken) {
        if (to == -1) {
            for (int i = 0; i < 24; i++) {
                Cell cell = board.getCell(i);
                if (cell != null && !cell.isEmpty() && cell.getColor() ==
playersColor) {
                    if (playersColor == Cell.WHITE && i < 18)
                        return false;
                    if (playersColor == Cell.BLACK && (i < 6 || i > 11))
                        return false;
                }
            }

            if (playersColor == Cell.WHITE && (from + diceValue < 24)) return
false;
            if (playersColor == Cell.BLACK && (from + diceValue < 12)) return
false;

            return true;
        }

        if (from < 0 || to < 0 || from >= 24 || to >= 24) return false;

        if (!isDiceValueMatching(from, to, diceValue, playersColor)) return
false;

        Cell fromCell = board.getCell(from);
        Cell toCell = board.getCell(to);

        if (fromCell.isEmpty() || fromCell.getColor() != playersColor) return
false;

        if (!toCell.isEmpty() && toCell.getColor() != playersColor) return
false;

        if (isHeadPosition(from, playersColor) && headMoveDone) {
            if (isFirstGameMove){

```

```

        boolean isBlockDouble = (diceValue == 3 || diceValue == 4 ||
diceValue == 6);

        if (isDouble && isBlockDouble) {
            if (headPiecesTaken < 2) return true;
        }

        return false;
    }

    return true;
}

public void move(Board board, int from, int to) {
    board.movePieces(from, to);
}

private boolean isHeadPosition(int position, int playersColor) {
    if (playersColor == Cell.WHITE && position == 0) return true;
    if (playersColor == Cell.BLACK && position == 12) return true;

    return false;
}

private boolean isDiceValueMatching(int from, int to, int diceValue, int
playersColor) {
    int distance;

    if (playersColor == Cell.WHITE)
        distance = to - from;

    else if (playersColor == Cell.BLACK) {
        if (to >= from)
            distance = to - from;
        else
            distance = (24 - from) + to;
    } else
        return false;
}

```

```

        return distance == diceValue;
    }

    private boolean isBlock(Board board, int from, int to, int playersColor) {
        Cell fromCell = board.getCell(from);
        Cell toCell = board.getCell(to);

        fromCell.removePiece();
        toCell.addPiece(playersColor);

        boolean hasSixBlock = false;
        int blockStartID = -1;

        for (int i = 0; i < 24; i++) {
            int continuousPieces = 0;
            for (int j = 0; j < 6; j++) {
                int checkID = (i + j) % 24;
                if (board.getCell(checkID).getColor() == playersColor)
                    continuousPieces++;
                else
                    break;
            }

            if (continuousPieces == 6) {
                hasSixBlock = true;
                blockStartID = i;
                break;
            }
        }

        boolean isMoveLegal = true;

        if (hasSixBlock) {
            int opponentColor = (playersColor == Cell.WHITE) ? Cell.BLACK :
Cell.WHITE;

            boolean opponentAhead = isOpponentAheadOfBlock(board,
blockStartID, playersColor, opponentColor);

            if (!opponentAhead)
                isMoveLegal = false;
        }
    }

```

```

        toCell.removePiece();
        fromCell.addPiece(playersColor);

        return isMoveLegal;

    }

    private boolean isOpponentAheadOfBlock(Board board, int blockStartID, int
playersColor, int opponentColor) {
        //проверка ячеек по кругу начиная со следующей после блока
        for (int i = 6; i < 24; i++) {
            int checkID = (blockStartID + i) % 24;
            Cell cell = board.getCell(checkID);

            //если найдена шашка перед блоком
            if (cell.getColor() == opponentColor && cell.getCount() > 0) {
                //если шашка в своем доме
                if (opponentColor == Cell.WHITE && (checkID >= 18 && checkID <=
23))

                    continue;

                if (opponentColor == Cell.BLACK && (checkID >= 6 && checkID <=
11))

                    continue;

                return true;
            }
        }

        return false;
    }

    private boolean areAllPiecesOnHouse(Board board, int playersColor) {
        if (playersColor == Cell.WHITE) {
            for (int i = 0; i < 17; i++) {
                Cell cell = board.getCell(i);
                if (cell.getColor() == Cell.WHITE && cell.getCount() > 0)
                    return false;
            }
        } else if (playersColor == Cell.BLACK) {

```

```

        for (int i = 0; i < 24; i++) {
            if (i >= 6 && i <= 11)
                continue;

            Cell cell = board.getCell(i);
            if (cell.getColor() == Cell.BLACK && cell.getCount() > 0)
                return false;
        }
    } else
        return false;

    return true;
}

public boolean canRemoveFromBoard(Board board, int playersColor, int from,
int diceValue) {
    if (!areAllPiecesOnHouse(board, playersColor)) return false;

    Cell cell = board.getCell(from);
    if (cell.isEmpty() || cell.getColor() != playersColor) return false;

    int position;
    if (playersColor == Cell.WHITE)
        position = 24 - from;
    else
        position = 12 - from;

    if (position == diceValue) return true;

    if (diceValue > position) {
        if (playersColor == Cell.WHITE) {
            for (int i = 18; i < from; i++) {
                if (board.getCell(i).getColor() == Cell.WHITE &&
board.getCell(i).getCount() > 0)
                    return false;
            }
        }

        } else {
            for (int i = 6; i < from; i++) {
                if (board.getCell(i).getColor() == Cell.WHITE &&
board.getCell(i).getCount() > 0)
                    return false;
            }
        }
    }
}

```

```

        }
    }

    return true;
}

return false;
}
}

```

Файл Oyn.java

```

package ru.psu.coursework.additional.logic;

public class Oyn implements TypeOfWin {
    public String getName() {
        return "Oyn";
    }

    public int getScorePoints() {
        return 1;
    }

    public String getDescription(String player) {
        return "Player " + player + " won!";
    }
}

```

Файл TypeOfWin.java

```

package ru.psu.coursework.additional.logic;

public interface TypeOfWin {
    String getName();
    int getScorePoints();
    String getDescription(String winner);
}

```

Файл WinConditionChecker.java

```

package ru.psu.coursework.additional.logic;

import ru.psu.coursework.additional.models.*;

```

```

public class WinConditionChecker {

    public WinConditionChecker() {

    }

    private int countTotalPieces(Board board, int playersColor) {
        int total = 0;

        for (int i = 0; i < 24; i++) {
            Cell cell = board.getCell(i);
            if (cell != null && cell.getColor() == playersColor) {
                total += cell.getCount();
            }
        }

        return total;
    }

    public boolean isGameOver(Board board) {
        return countTotalPieces(board, Cell.WHITE) == 0 ||
countTotalPieces(board, Cell.BLACK) == 0;
    }

    public int getWinnerColor(Board board) {
        if (countTotalPieces(board, Cell.WHITE) == 0) return Cell.WHITE;

        if (countTotalPieces(board, Cell.BLACK) == 0) return Cell.BLACK;

        return Cell.EMPTY;
    }

    public TypeOfWin checkWin(Board board, int winnersColor) {
        int losersColor = (winnersColor == Cell.WHITE) ? Cell.BLACK :
Cell.WHITE;
        int loserPiecesLeft = countTotalPieces(board, losersColor);

        if (loserPiecesLeft == 15) return new Mars();

        return new Oyn();
    }
}

```

```
    }  
}
```

Файл Commands.java

```
package ru.psu.coursework.additional.messaging;
```

```
public interface Commands {  
    int REGISTRATION_REQUEST = 1;  
    int LOGIN_REQUEST = 2;  
    int AUTHENTICATION_SUCCESS = 3;  
    int CREATE_ROOM = 4;  
    int JOIN_ROOM = 5;  
    int GAME_START = 6;  
    int DICE_ROLL = 7;  
    int MAKE_MOVE = 8;  
    int UPDATE_BOARD = 9;  
    int ERROR = 0;  
  
}
```

Файл Board.java

```
package ru.psu.coursework.additional.models;
```

```
import java.io.Serializable;  
import java.util.ArrayList;  
import java.util.List;
```

```
public class Board implements Serializable {  
    private static final int COUNT_OF_CELLS = 24;  
  
    private final List<Cell> cells;  
  
    public Board() {  
        cells = new ArrayList<>(COUNT_OF_CELLS);  
  
        for (int i = 0; i < COUNT_OF_CELLS; i++) {  
            cells.add(new Cell());  
        }  
  
        setupPosition();  
    }  
}
```



```

public void setupPosition() {

    cells.get(0).setCount(15);
    cells.get(0).setColor(Cell.WHITE);

    cells.get(12).setCount(15);
    cells.get(12).setColor(Cell.BLACK);

}

public List<Cell> getCells() {
    return cells;
}

public Cell getCell(int id){
    if (id < 0 || id >= 24) {
        return null;
    }

    return cells.get(id);
}

public void movePieces(int from, int to) {
    Cell fromCell = cells.get(from);
    Cell toCell = cells.get(to);

    int moverColor = fromCell.getColor();

    fromCell.removePiece();
    toCell.addPiece(moverColor);

}
}

```

Файл Cell.java

```

package ru.psu.coursework.additional.models;

import java.io.Serializable;

public class Cell implements Serializable {

```

```

public static final int EMPTY = 0;
public static final int WHITE = 1;
public static final int BLACK = 2;

private int count;
private int color;

public Cell() {
    this.count = 0;
    this.color = EMPTY;
}

public boolean isEmpty() {
    return count == 0;
}

public void addPiece(int pieceColor){
    if (this.color != EMPTY && this.color != pieceColor)
        throw new IllegalArgumentException("Error!!!");

    this.color = pieceColor;
    this.count++;
}

public void removePiece(){
    if (count > 0) {
        count--;
        if (count == 0) color = EMPTY;
    }
}

public int getCount() {
    return count;
}

public void setCount(int count) {
    this.count = count;
}

```

```

    public int getColor() {
        return color;
    }

    public void setColor(int color) {
        this.color = color;
    }

}

```

Файл Dices.java

```

package ru.psu.coursework.additional.models;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class Dices implements Serializable {
    private int diceOne;
    private int diceTwo;
    private int diceOneUses;
    private int diceTwoUses;

    private Random generator;

    public Dices() {
        generator = new Random();
    }

    public boolean isDouble = false;

    public void roll(){
        diceOne = generator.nextInt(6) + 1;
        diceTwo = generator.nextInt(6) + 1;

        if (diceOne == diceTwo) {
            diceOneUses = diceTwoUses = 2;
            isDouble = true;
        }
        else

```

```

        diceOneUses = diceTwoUses = 1;

    }

    public void takeDice(int number){
        if (diceOneUses > 0 && diceOne == number)
            diceOneUses--;
        else if (diceTwoUses > 0 && diceTwo == number)
            diceTwoUses--;
        else
            throw new IllegalArgumentException("Invalid dice!!!");
    }

    public int takeDiceOne(){
        if (diceOneUses == 0) return 0;

        diceOneUses--;

        return diceOne;
    }

    public int takeDiceTwo(){
        if (diceTwoUses == 0) return 0;

        diceTwoUses--;

        return diceTwo;
    }

    public boolean isRolled(){
        return diceOneUses > 0 || diceTwoUses > 0;
    }

    public int getDiceOne(){
        return diceOne;
    }

    public int getDiceTwo(){
        return diceTwo;
    }

    public List<Integer> getAvailableValues() {

```

```

        List<Integer> available= new ArrayList<>();

        for (int i = 0; i < diceOneUses; i++)
            available.add(diceOne);

        for (int i = 0; i < diceTwoUses; i++)
            available.add(diceTwo);

        return available;
    }

    public void clear() {
        this.diceOne = 0;
        this.diceTwo = 0;
        this.diceOneUses = 0;
        this.diceTwoUses = 0;
        this.isDouble = false;
    }
}

```

Приложение Б. UML-диаграммы

UML – диаграмма вариантов использования приложения

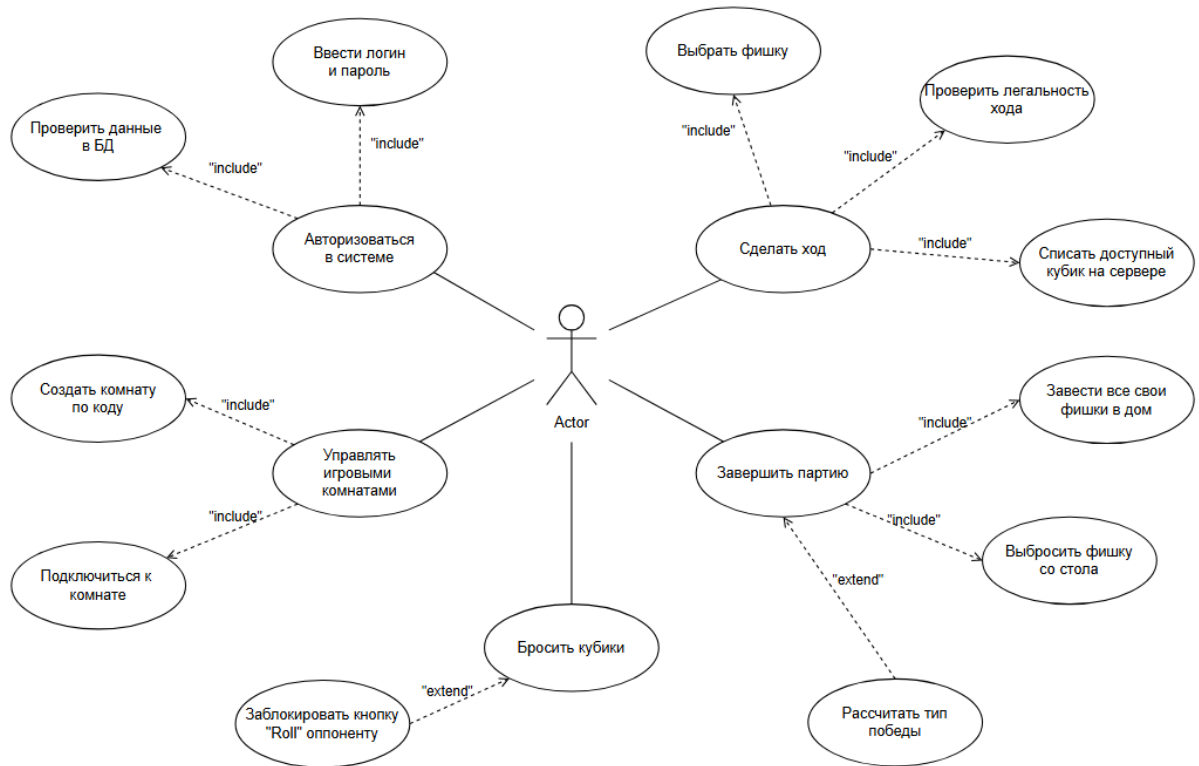


Рисунок Б1 - UML – диаграмма вариантов использования приложения

UML – диаграмма деятельности

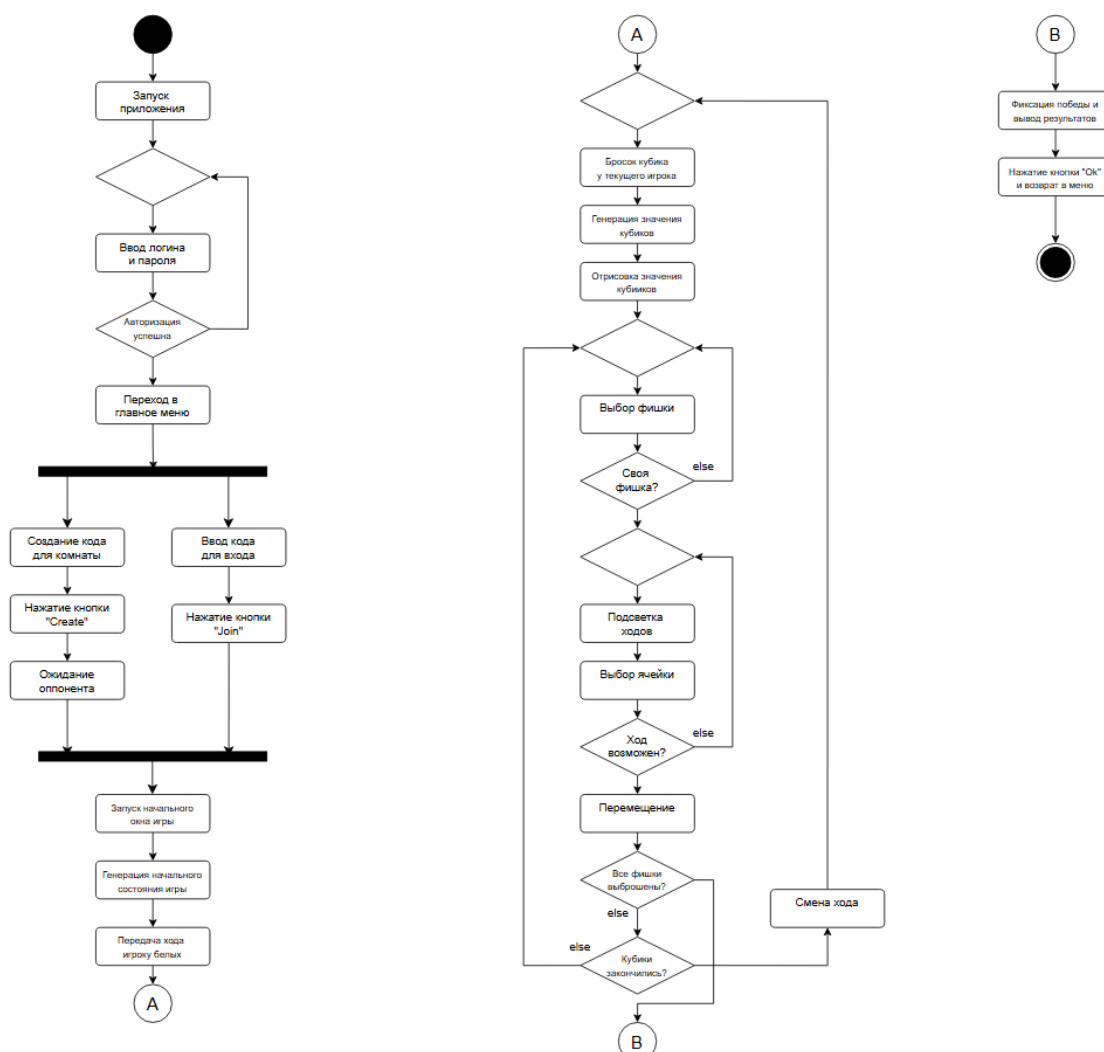


Рисунок Б2 - UML – диаграмма деятельности

UML-диаграмма развертывания

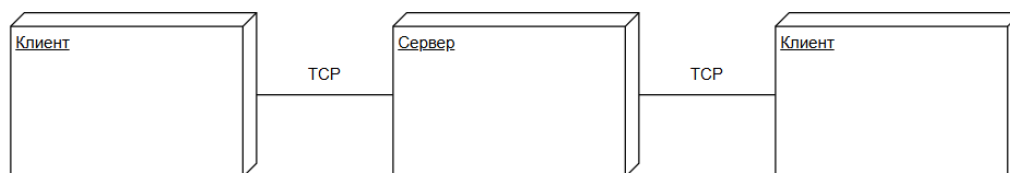


Рисунок Б3 - UML-диаграмма развертывания

UML-диаграмма последовательности

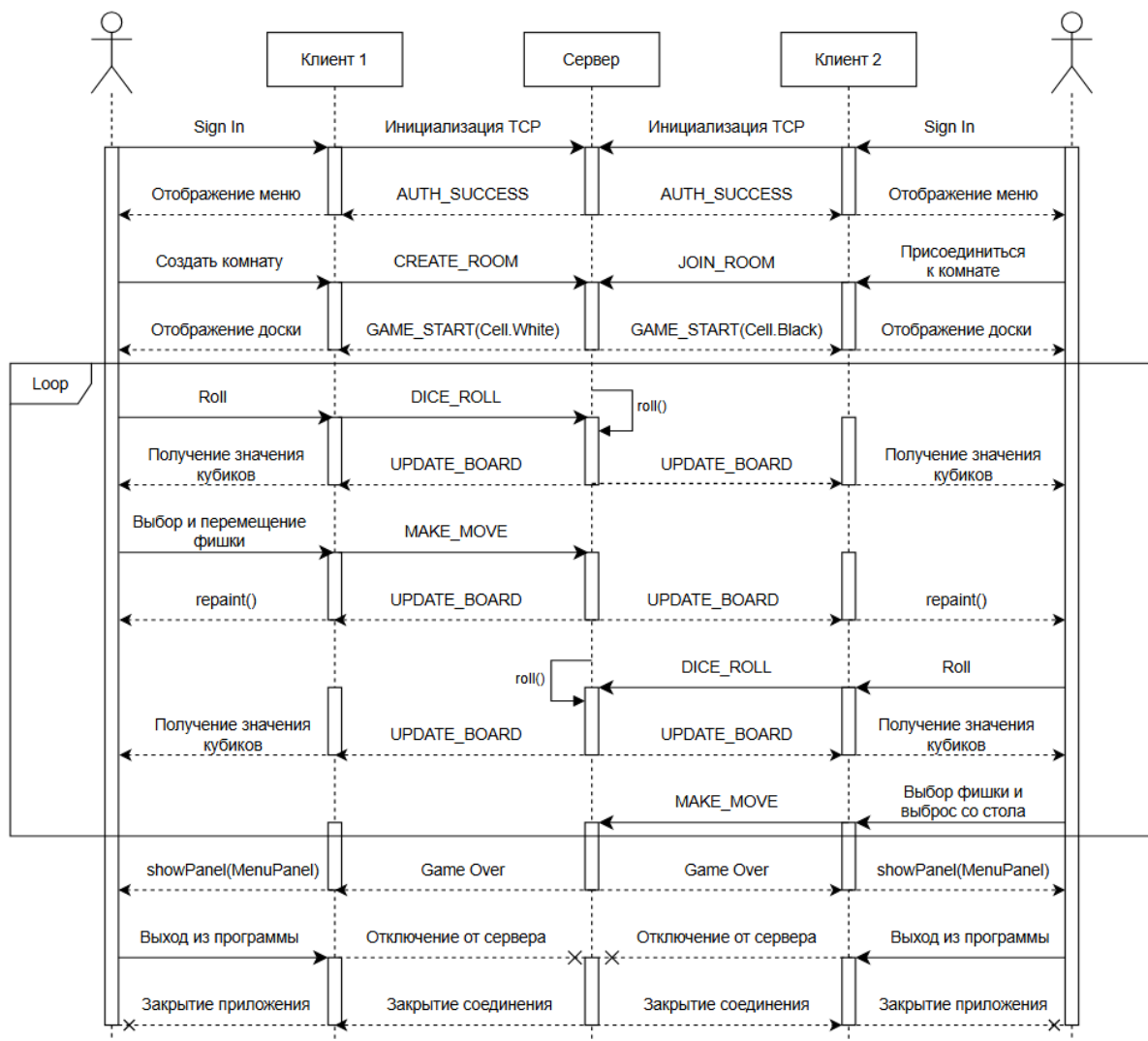


Рисунок Б4 - UML-диаграмма последовательности

UML-диаграмма классов клиента

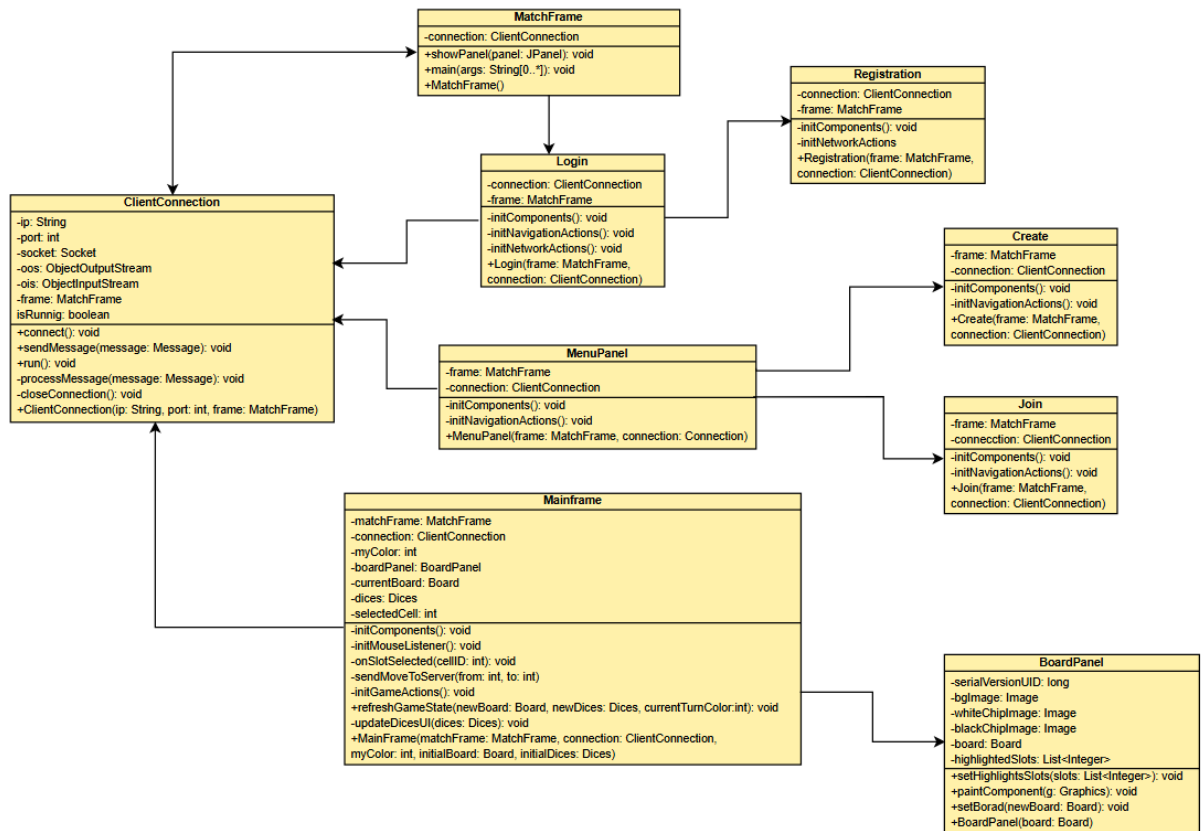


Рисунок 20 - UML-диаграмма классов клиента

UML-диаграмма классов сервера

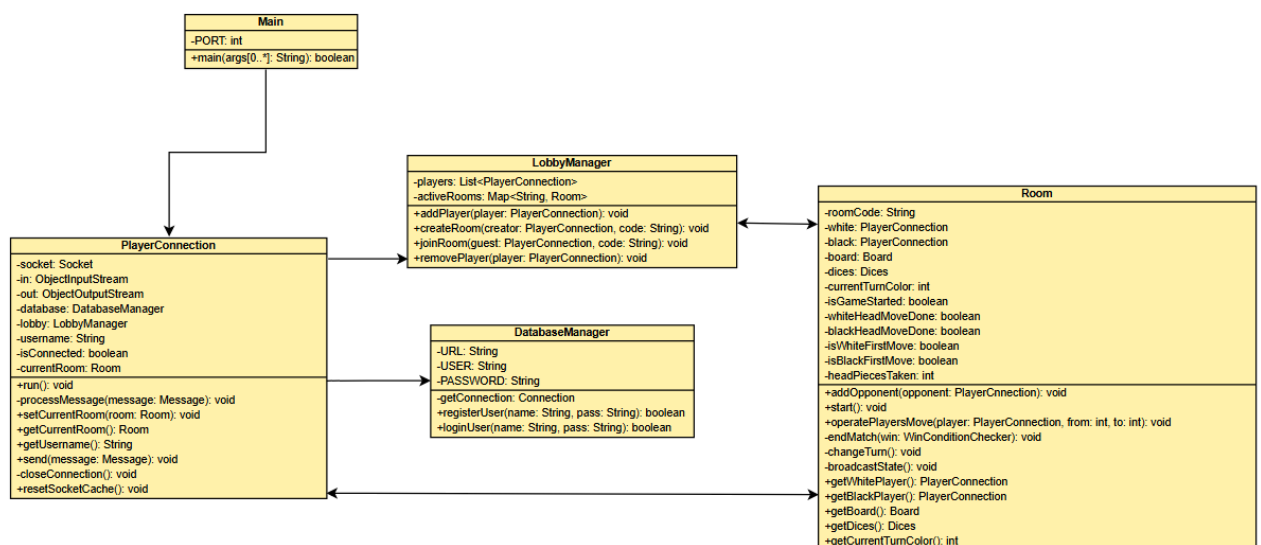


Рисунок Б6 - UML-диаграмма классов сервера